

INDEX

S.No	EXPERIMENT	SIGNATURE
1.	Introduction to PROTEUS and MPLAB.	
2.	Introduction to PIC Microcontroller.	
3.	Write a Program to blink a set of 8 LEDs interfaced with a PIC Microcontroller and verify its working using proteus such that all 8 LEDs blink simultaneously.	
4.	Write a Program to blink a set of 8 LEDs interfaced with a PIC microcontroller and verify its working using proteus such that all 8 LEDs blink alternately.	
5.	Write a Program to blink a set of 8 LEDs interfaced with a PIC Microcontroller and verify its working using proteus such that all 8 LEDs blink one by one.	
6.	Write a Program to display digit 0-9 on seven segment display interfaced with a PIC microcontroller and verify its working using proteus via follow configurations.	
7.	Write a Program to display A-C on 5X7 LED matrix display interfaced with a PIC microcontroller and verify its working using proteus.	
8.	Write a Program to display 2 digit number on 8x8 LED matrix display interfaced with a PIC microcontroller and verify its working using proteus.	

EXPERIMENT-1

AIM: Introduction to PROTEUS AND MPLAB.

SOFTWARE: PROTEUS 8 PROFESSIONAL, MPLAB X IDE V5.10

THEORY:

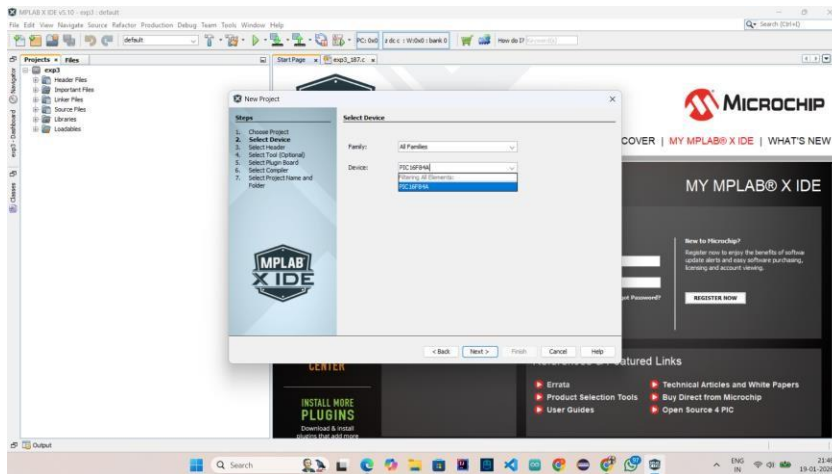
MPLAB: MPLAB® X Integrated Development Environment (IDE) is a software program that runs on a PC (Windows, Mac OS®, Linux) to develop applications for Microchip microcontrollers and digital signal controllers. It is called an Integrated Development Environment because it provides a single integrated "environment" to develop code for embedded microcontrollers.

MPLAB X IDE brings a host of features to enhance the debug experience during the design phase of your project. Based on the open source NetBeans IDE from Oracle, Microchip has been able to add many frequently requested features very quickly and easily, while also providing a much more extensible architecture to bring you even more new features fast - including added support for PIC®, dsPIC®, AVR, CEC and SAM microcontrollers and digital signal controllers

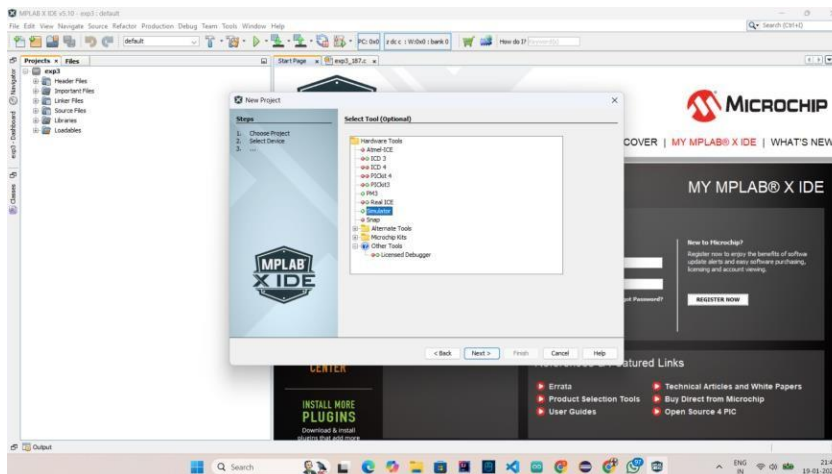
MPLAB IDE runs as a 32-bit application on MS Windows, is easy to use and includes a host of free software components for fast application development and super-charged debugging MPLAB IDE also serves as a single, unified graphical user interface for additional Microchip and third party software and hardware development tools. Moving between tools is a snap, and upgrading from the free software simulator to hardware debug and programming tools is done in a flash because MPLAB IDE has the same user interface for all tools

PROTEUS: Many CAD users dismiss schematic capture as a necessary evil in the process of creating PCB layout but we have always disputed this point of view. With PCB layout now offering automation of both component placement and track routing, getting the design into the computer can often

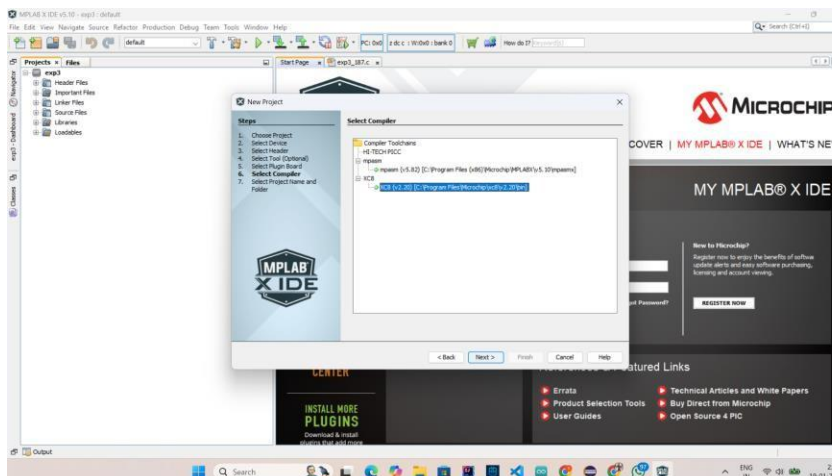
3. Select the device PIC16F84A, click on next



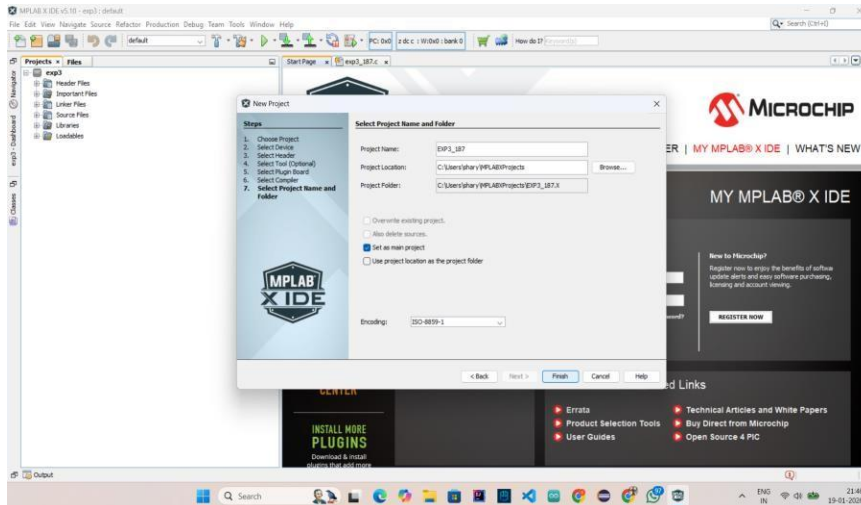
4. Select tool simulator, click on next



5. Now select the compiler, click on XC8 then click on next

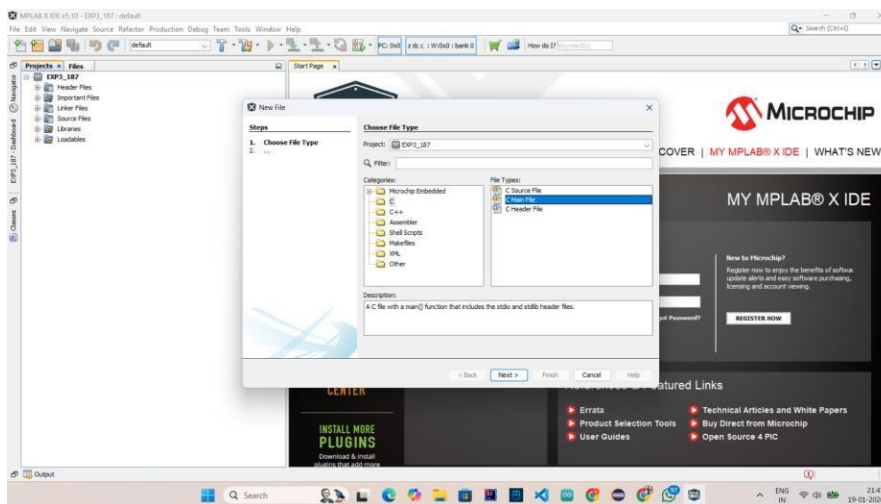


6. Add project name (Set as main project checked), click on next

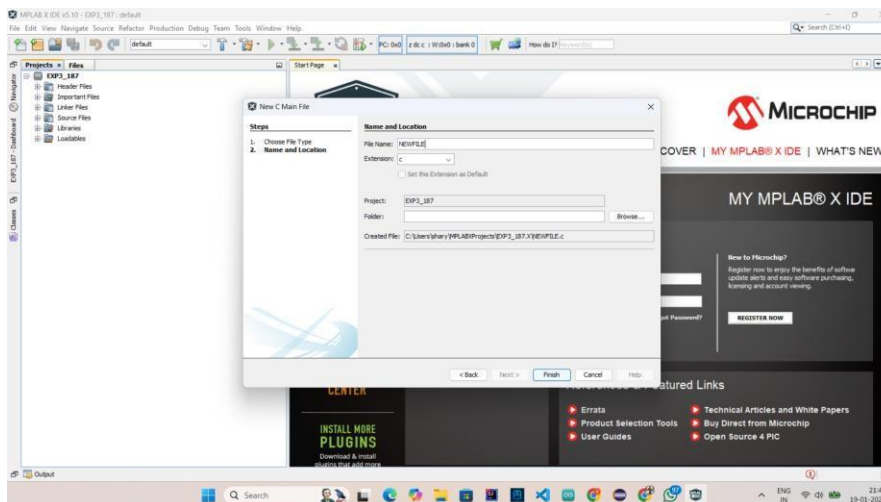


7. Now click on new file

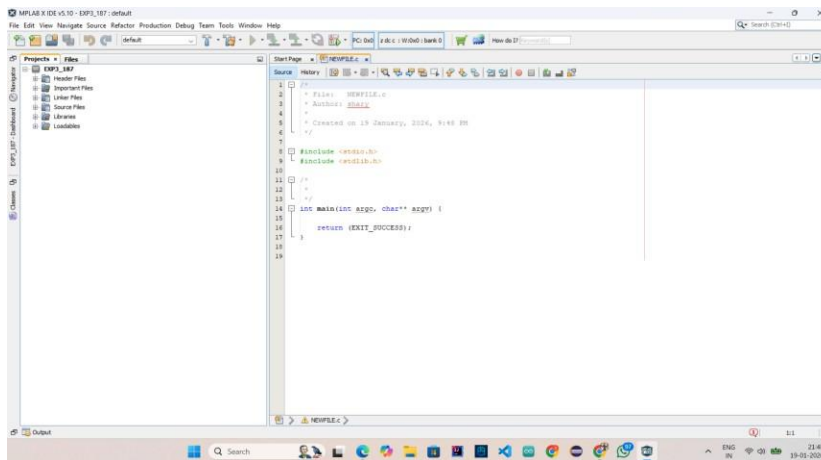
8. Select C then, C MAIN FILE



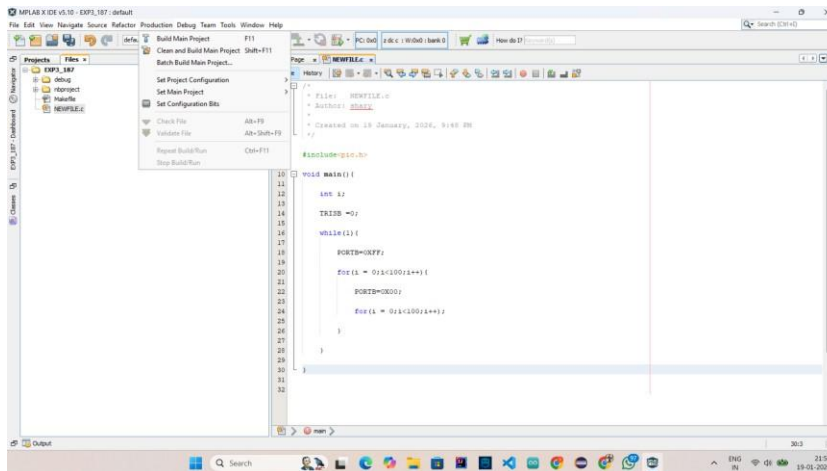
9. Add file name and location(optional)



10. Write the code



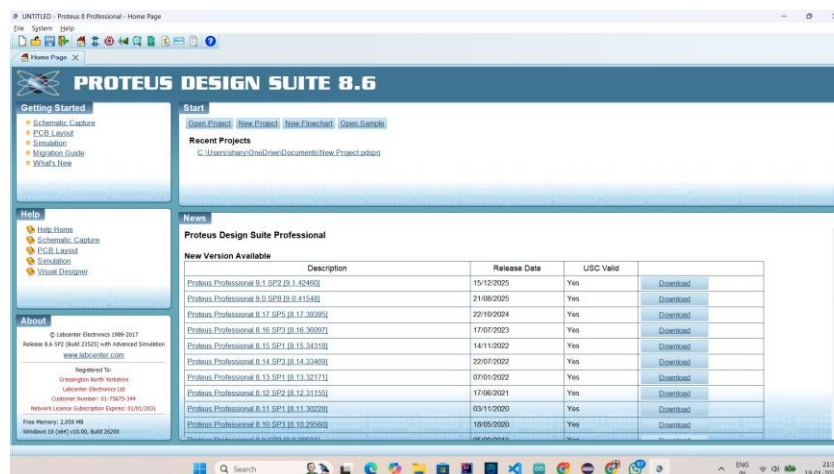
11. Save the file then click on Build Main Project



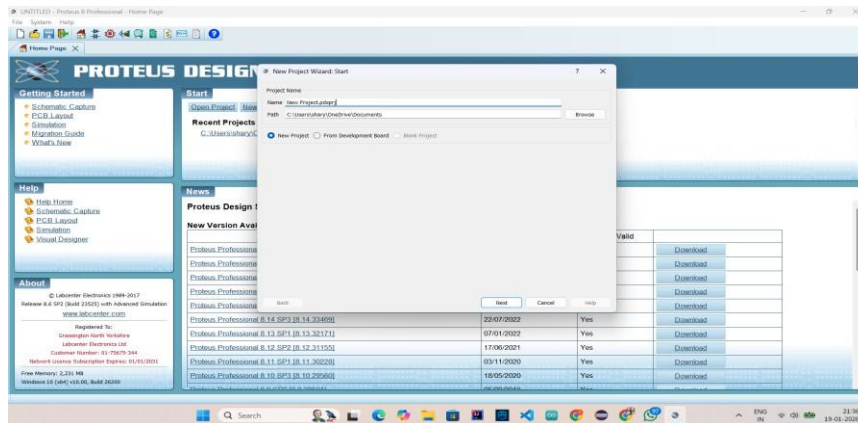
12. Hex file is created at the desired path

PROTEUS

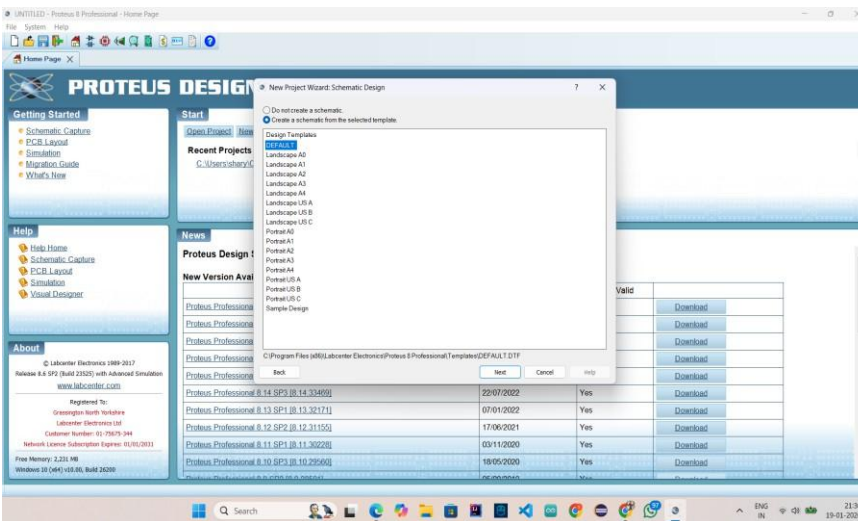
1. Click on New Project.



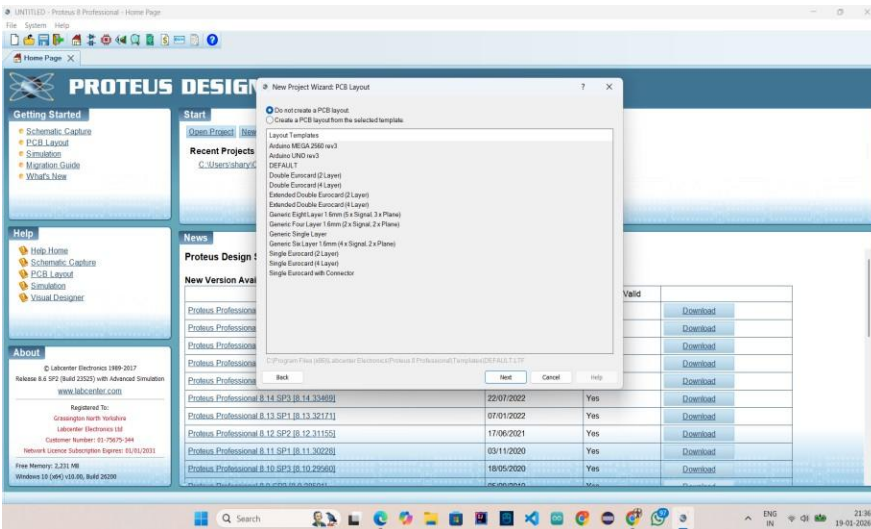
2. Add name of your project C select the folder where you want to store your project, click on next.



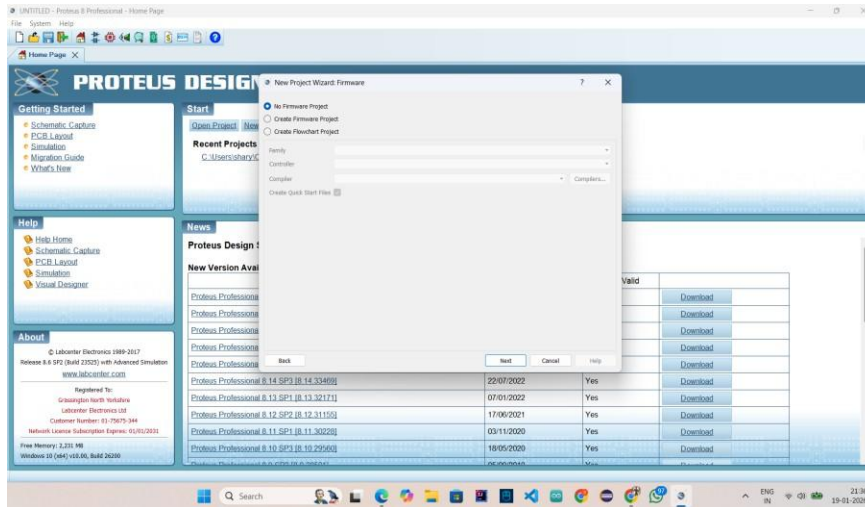
3. Select Option 'Create a Schematic from a Template, select Default', next.



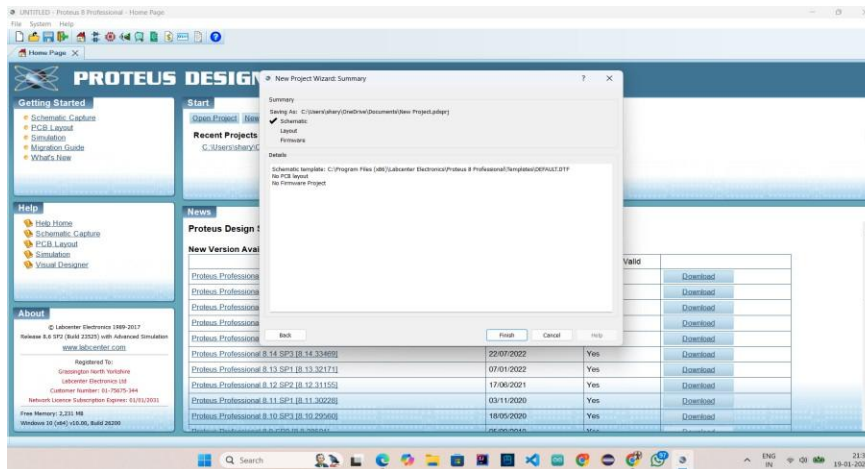
4. Select do not create a PCB layout, click on next.



5. Select no firmware project, click on next.



6. Click on finish.



7. Create the circuit.

8. Save the project.

9. Double click on controller, provide the path of hex file/program file, and then click on okay.

10. Run the project.

EXPERIMENT 2

AIM: Introduction to PIC Microcontroller

THEORY:

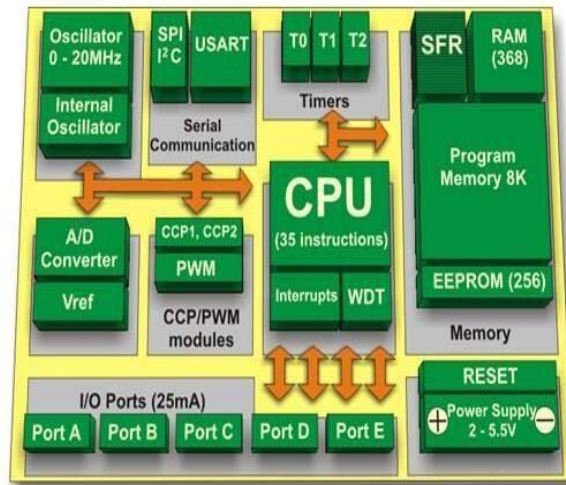
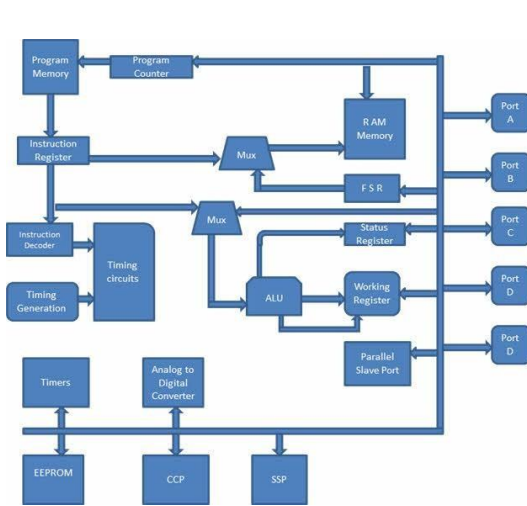
PIC MICROCONTROLLER:

Peripheral Interface Controller (PIC) is microcontroller developed by Microchip. PIC microcontroller is fast and easy to implement program when we compare other microcontrollers like 8051. The ease of programming and easy to interfacing with other peripherals PIC became successful microcontroller. PIC mainly used modified Harvard architecture and also supports RISC (Reduced Instruction Set Computer) by the above specification RISC and Harvard we can easily that PIC is faster than the 8051 based controller which is made-up of Von-Newman architecture.

PIC microcontroller from microchip are very popular microcontroller. PICs are easily programmable cheap microcontroller. PICs is the name for the microchip microcontroller family (peripheral interface controller). Consisting of a microprocessor, I/O ports, timer(s), and other internal integrated hardware. A wide range of chip sizes (from eight-pin up), great availability of compilers and source code and easy programming. Flash-type devices are re-programmable in-circuit, while OTP versions are very cheap to use at the final stage. A wide range of simple programmer hardware and software is downloadable from the net. PIC16f84 seems to be the "standard" for small projects. For the beginner with PICs, a PIC16184A or PIC166628 device is a good choice to start with. There are many different varieties of PICs that run at many frequencies, have different memory size, and different internal peripherals. The most famous MCU that microchip produces are: 12cxxx, 12fxxx, 16cxxx, 16fxxx, and 18fxxx.

- Prefix 12 is for chips with 8 pins.
- Prefix 16 is for 14-bit core chip with more than 8 pins.
- Prefix 18 is for 16-bit core chip.
- The letter after number tells the memory type:
 - C is for EPROM (OTP)
 - F is for flash chips
- The number (2 or 3 digits) after this letter identifies specific chip version.
- Improved new version of certain PICs types are identified by appending an (A)

ARCHITECTURE:



CPU:

CPU is not different from other microcontrollers CPU, PIC microcontroller CPU consists of Arithmetic logic unit (ALU), memory unit (MU), control unit (CU), Accumulator etc. We know that ALU mainly used for arithmetic operations and taking the logical decisions, memory used for storing the instruction which is to be processed and also storing the instructions after processing, Control unit is used for controlling all the peripherals which are connected to the CPU both internal peripherals and external peripherals.

Accumulator is used for storing the results and used for further processing. PIC micro controller supports the RISC architecture that is reduced instruction set computer, if a computer or controller is said that it supports reduced instruction set you should remember the following points:

1. RISC has very few instructions (approx. 35) which are used in the program.
2. Length of the instruction is small and fixed and takes same amount of time for processing.
3. As the instruction is small it will take less time to process another words CPU will be fast.
4. Compiler need not be complex and debugging will be very easy in the programmer point of view.

Memory: Memory module in the PIC consists of RAM, ROM and STACK RAM: RAM (Random Access Memory) which is a volatile memory used for storing the data temporarily in its registers. RAM memory is divided into Banks, in each bank we have number of registers. The RAM registers are divided into 2 types. They are General purpose registers (GPR) and Special purpose registers (SPR).

1. GPR:

General purpose registers as the name implies for general usage. For example if we want to multiply any two numbers using PIC we generally take two registers for storing the numbers and multiply the two numbers and store the result in other registers. So general purpose registers will not have any special function or any special permission, CPU can easily access the data in the registers.

2. SPR:

Special function registers are having the specific functions, when we use this register they will act according to the functions assigned to them. They cannot be used like normal registers. For example you cannot use STATUS register for storing the data, STATUS registers are used for showing the status of the program or operation. User cannot change the function of the Special function register, the function is given by the vendor at the manufacturing time.

ROM:

ROM (Read Only memory) is a nonvolatile memory used for storing the data permanently. In microcontroller ROM will store the complete instructions or program, according to the program microcontroller will act. Rom is also called program memory in this memory user will write the program for microcontroller and save it permanently and get executed by the CPU. According to the instruction executed by the CPU the PIC microcontroller will perform the task. In ROM there are different types which are used in different PIC microcontrollers

1. EEPROM:

In the ordinal ROM we can write the program for only one time we cannot reuse the Microcontroller for another time where as in the EEPROM (Electrically Erasable Programmable Read Only Memory) we can program the ROM for number of times.

2. Flash Memory:

Flash memory is also PROM in which we can read write and erase the program more than 10,000 times Mostly PIC microcontroller uses this type of ROM.

Stack: When an interrupt occur PIC has to first execute the interrupt and the existing process address which is being executed is stored in the stack. After completing the interrupt execution, PIC will call the process with the help of address which is stored in stack and get executing the process.

Bus: Bus is mainly used for transferring and receiving the data from one peripheral to another. There are two types of buses.

1. **Data Bus:** It is used to transfer/receive only the data.

2. **Address Bus:** is used to transmit the memory address from peripherals to CPU. 10 pins are used for interfacing the external peripherals, UART and USART is serial communication protocol which is used for interfacing serial devices like GPS, GSM, IR, Bluetooth etc.

Advantages of PIC Microcontroller:

1. They are reliable and malfunctioning of PIC percentage is very less. And performance of the PIC is very fast because of using RISC architecture.
2. Power conception is also very less when compared to other micro controllers. When we see in the programmer point of view interfacing is very easy, also we can connect analog devices directly without any extra circuitry and use them. Programming is also very easy when compared to other microcontrollers.
3. Small instruction set to learn.
4. RISC architecture.
5. Built-in oscillator with selectable speeds.
6. Easy entry level, in-circuit programming plus in-circuit debugging PIC kit units available for less than \$50.
7. Inexpensive microcontrollers.
8. Availability of processors in DIL package make them easy to handle for hobby use.

Disadvantages of PIC Microcontroller:

1. The length of the program will be big because of using RISC (35 instructions).
2. Program memory is not accessible and only one single accumulator is present
3. One accumulator.
4. Register bank switching is required to access the entire RAM of many devices.
5. Operations C registers are not orthogonal, some instructions can address RAM or/and immediate constants, while others can use the accumulator only.

PIC FAMILIES:

1. The first family, PIC10 (16FXXX) is called Low End.

The PIC10FXXX devices from Microchip Technology are low-cost, high high-performance, 8-bit, fully static, Flash-based CMOS microcontrollers. They employ a RISC architecture with only 33 single-word/ single-cycle instructions. The 12-bit wide instructions are highly symmetrical. The easy-to-use and easy to remember instruction set reduces development time significantly. The PIC10FXXX devices contain an 8-bit ALU and working register

2. The second family, PIC12 (PIC12FXXX) -- is called Mid-Range. The PIC12FXXX most popular among these starter their way in this field. Mid-Range devices feature 14-bit program word architecture and are available in 8 to 64-pin packages that offer an operating voltage range of 1.8-5.5V, small package footprints, interrupt handling, an 8-level hardware stack, multiple A/D channels and EEPROM data memory. Mid-range devices offer a wide range of package options and a wide range of peripheral integration. These devices feature various serial analog and digital peripherals, such as: SPI, I2C, USART, LCD and A/D converters

3. The third family is PIC16 (16FXXX)

With six variants ranging from 3.5K-14 Kbytes of Flash memory, up to 256 bytes of RAM and a mix of peripherals including USART, CCP and onboard analog comparators. These devices are well suited for designers with applications that need more code space or I/O than 14-pin variants supply, and are looking to increase system performance and code efficiency by employing hardware motor control and communications capability.

4. The fourth family is PIC 17/18(18FXXX).

The PIC18 family utilizes a 16-bit program word architecture and incorporates an advanced RISC architecture with 32 level-deep stack, 8x8 hardware multiplier, and multiple internal and external interrupts. With the highest performance in Microchip's 8-bit portfolio, the PIC18 family provides up to 16 MIPS and linear memory. PIC18 is the most popular architecture for new 8-bit designs where customers want to program in C language.

INSTRUCTION SET:

Complete set which includes 35 instructions is given in the following table. A reason for such a small number of instructions lies primarily in the fact that we are talking about a RISC microcontroller whose instructions are well optimized considering the speed of work, architectural simplicity and code compactness The only drawback is that programmer is expected to master "uncomfortable" technique of using a reduced set of 35 instructions.

Data transfer:

Transfer of data in a microcontroller is done between work (W) register and any register that represents any location in internal RAM (regardless whether those are special or general purpose registers). First these instructions (look at the following table) provide for a constant being written in W register

(MOVLW is short for Move Literal to W), and for data to be copied from W register onto RAM and data from RAM to be copied onto W register for on the same RAM location, at which point only the status of Z flag changes, Instruction CLRF writes constant 0 in 'f' register, and CLRW writes constant 0 in register W. SWAPF instruction exchanges places of the 4-bit nibbles field inside a register.

Arithmetic and logic: Of all arithmetic operations, PIC like most microcontrollers supports only subtraction and addition. Flags C, DC and Z are set depending on a result of addition or subtraction, but with one exception: since subtraction is performed like addition of a negative value, C flag is inverse following a subtraction. In other words, it is set if operation is possible, and reset if larger number was subtracted from a smaller one.

Logic unit of PIC has capability of performing operations AND, OR, EX-OR, complementing (COMF) and rotation (RLF and RRF). Instructions which rotate the register contents move bits inside a register through flag C by one space to the left (toward bit 7), or to the right (toward bit 0). Bit which "comes out" of a register is written in flag C, and value of C flag is written in a bit on the "opposite side" of the register.

Bit operations: Instructions BCF and BSF do setting or cleaning of one bit anywhere in the memory. Even though this seems like a simple operation, it is executed so that CPU first reads the whole byte, changes one bit in it and then writes in the entire byte at the same place.

Word list:

f any memory location in a microcontroller

W work register

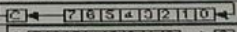
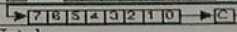
b bit position in 'f' register

d destination bit label group of eight characters which marks the beginning of a part of the program

TOS top of stack

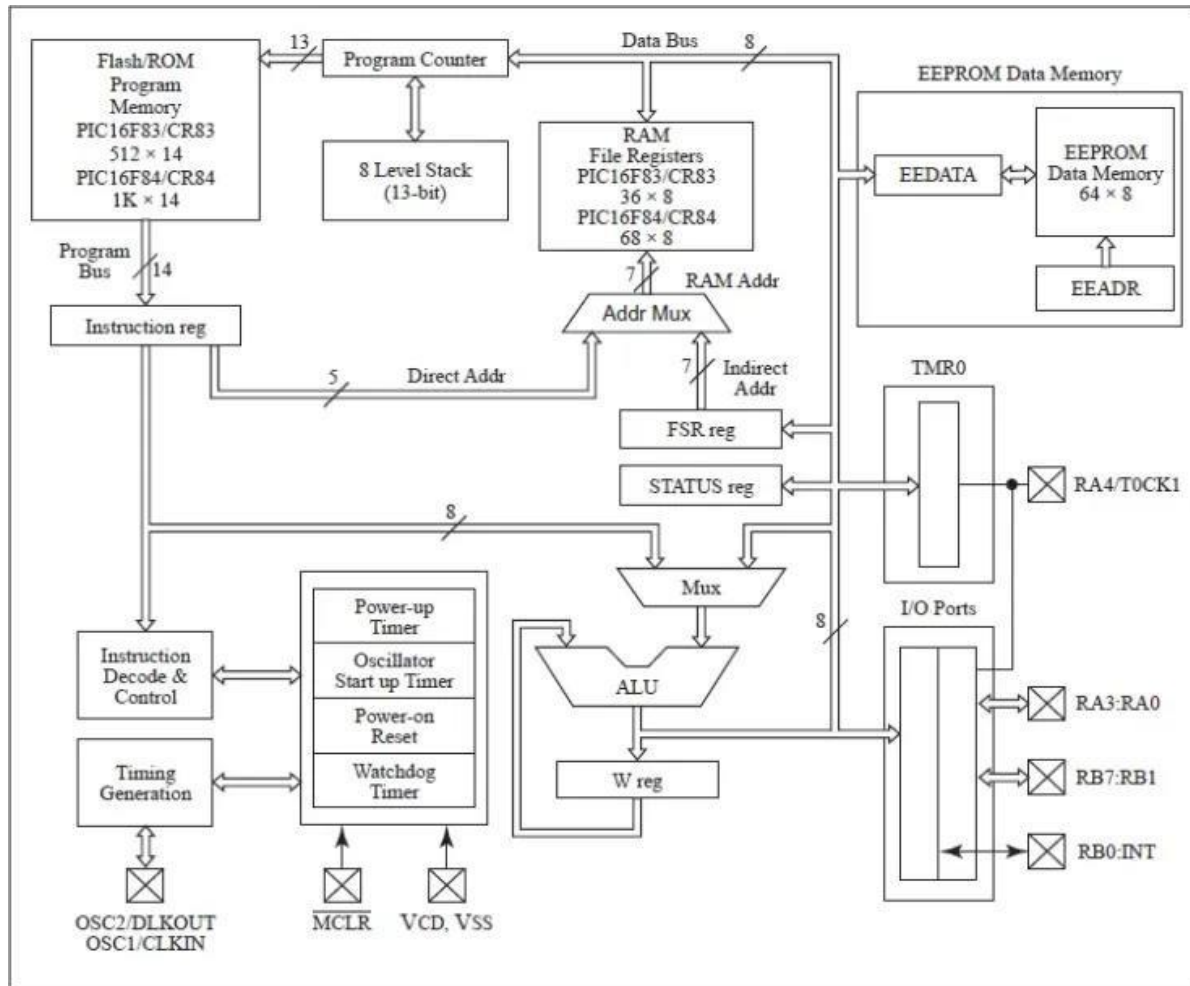
[] option

<> bit position inside register

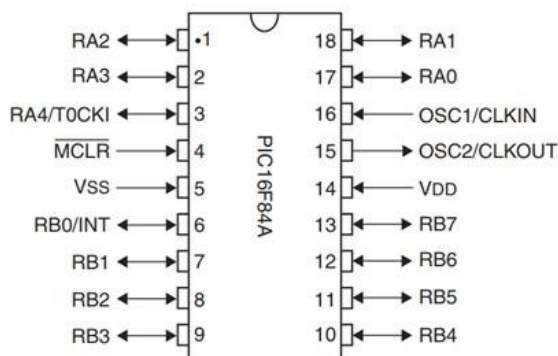
Mnemonic		Description	Operation	Flag	Cycle	Notes
Data transfer						
MOVLW	k	Move constant to W	$k \rightarrow W$		1	
MOVWF	f	Move W to f	$W \rightarrow f$		1	
MOVF	f, d	Move f	$f \rightarrow d$	Z	1	1,2
CLRW	-	Clear W	$0 \rightarrow W$	Z	1	
CLRF	f	Clear f	$0 \rightarrow f$	Z	1	2
SWAPF	f, d	Swap nibbles in f	$f(7:4), (3:0) \rightarrow f(3:0), (7:4)$		1	1,2
Arithmetic and logic						
ADDLW	k	Add constant and W	$W+1 \rightarrow W$	C,DC,Z	1	
ADDWF	f, d	Add W and f	$W+f \rightarrow d$	C,DC,Z	1	1,2
SUBLW	k	Subtract W from constant	$W-k \rightarrow W$	C,DC,Z	1	
SUBWF	f, d	Subtract W from f	$W-f \rightarrow d$	C,DC,Z	1	1,2
ANDLW	k	AND constant with W	$W.AND.k \rightarrow W$	Z	1	
ANDWF	f, d	AND W with f	$W.AND.f \rightarrow d$	Z	1	1,2
IORLW	k	OR constant with W	$W.OR.k \rightarrow W$	Z	1	
IORWF	f, d	OR W with f	$W.OR.f \rightarrow d$	Z	1	1,2
XORLW	k	Exclusive OR constant with W	$W.XOR.k \rightarrow W$	Z	1	1,2
XORWF	f, d	Exclusive OR W with f	$W.XOR.f \rightarrow d$	Z	1	
INCF	f, d	Increment f	$f+1 \rightarrow f$	Z	1	1,2
DECF	f, d	Decrement f	$f-1 \rightarrow f$	Z	1	1,2
RLF	f, d	Rotate Left f through carry		C	1	1,2
RRF	f, d	Rotate Right f through carry		C	1	1,2
COMF	f, d	Complement f	$f \rightarrow d$	Z	1	1,2
Bit operations						
BCF	f, b	Bit Clear f	$0 \rightarrow f(b)$		1	1,2
BSF	f, b	Bit Set f	$1 \rightarrow f(b)$		1	1,2
Directing a program flow						
BTFSC	f, b	Bit Test f, Skip if Clear	jump if $f(b)=0$		1 (2)	3
BTFSS	f, b	Bit Test f, Skip if Set	jump if $f(b)=1$		1 (2)	3
DECFSZ	f, d	Decrement f, Skip if 0	$f-1 \rightarrow d$, jump if $Z=1$		1(2)	1,2,3
INCFSZ	f, d	Increment f, Skip if 0	$f+1 \rightarrow d$, jump if $Z=0$		1(2)	1,2,3
GOTO	k	Go to address	$W.AND.k \rightarrow W$		2	
CALL	k	Call subroutine	$W.AND.f \rightarrow d$		2	
RETURN	-	Return from Subroutine	$W.OR.k \rightarrow W$		2	
RETLW	k	Return with constant in W	$W.OR.f \rightarrow d$		2	
RETFIE	-	Return from interrupt	$W.XOR.k \rightarrow W$		2	
Other instructions						
NOP	-	No Operation			1	
CLRWDI	-	Clear Watchdog Timer	$0 \rightarrow WDT, 1 \rightarrow TO, 1 \rightarrow PD$	$\overline{TO}, PD$	1	
SLEEP	-	Go into standby mode	$0 \rightarrow WDT, 1 \rightarrow TO, 0 \rightarrow PD$	$\overline{TO}, \overline{PD}$	1	

PIC16F84A:

INTERNAL ARCHITECTURE:



PIN DIAGRAM:



PIC16F84 Microcontroller

PIN No.	PIN Name	Description
1	RA2	Second pin on port A. Has no additional function.
2	RA3	Third pin on port A. Has no additional function.
3	RA4	Fourth pin on port A. Tockl.
4	MCLR	Reset input and Vpp programming voltage of microcontroller.
5	Vss	Ground of power supply.
6	RB0	Zero pin on port B. Interrupt input is additional function.
7	RB1	First pin on port B. Has no additional function.
8	RB2	Second pin on port B. Has no additional function.
9	RB3	third pin on port B. Has no additional function.
10	RB4	Fourth pin on port B. Has no additional function.
11	RB5	Fifth pin on port B. Has no additional function.
12	RB6	Sixth pin on port B. 'clock' line in program mode.
13	RB7	Seventh pin on port B. 'Data' line in program mode.
14	Vdd	Positive power supply pole.
15	OSC2	Pin assigned for connecting with an oscillator.
16	OSC1	Pin assigned for connecting with an oscillator.
17	RA0	Zero pin on port A. Has no additional function.
18	RA1	First pin on port A. Has no additional function.

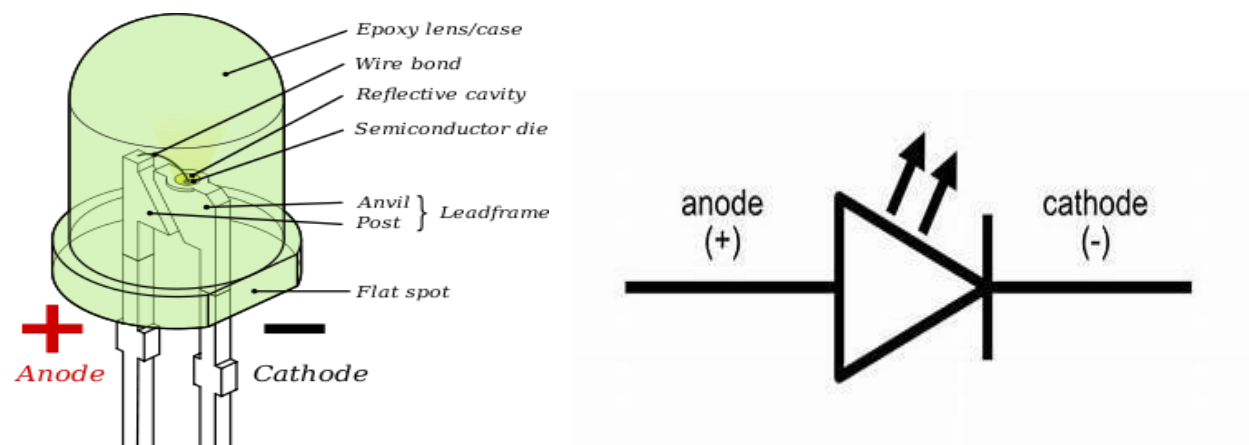
EXPERIMENT-3

AIM: To write an Embedded-C code to blink a set of 8 LEDs interfaced with a PIC microcontroller and verify its working using Proteus such that all 8 LEDs blink simultaneously.

SOFTWARE: MPLAB X IDE and Proteus 8

THEORY:

A light-emitting diode (LED) is a semiconductor light source that emits light when current flows through it. Electrons in the semiconductor recombine with electron holes, releasing energy in the form of photons. This effect is called electroluminescence. The color of the light (corresponding to the energy of the photons) is determined by the energy required for electrons to cross the band gap of the semiconductor. White light is obtained by using multiple semiconductors or a layer of light-emitting phosphor on the semiconductor device.

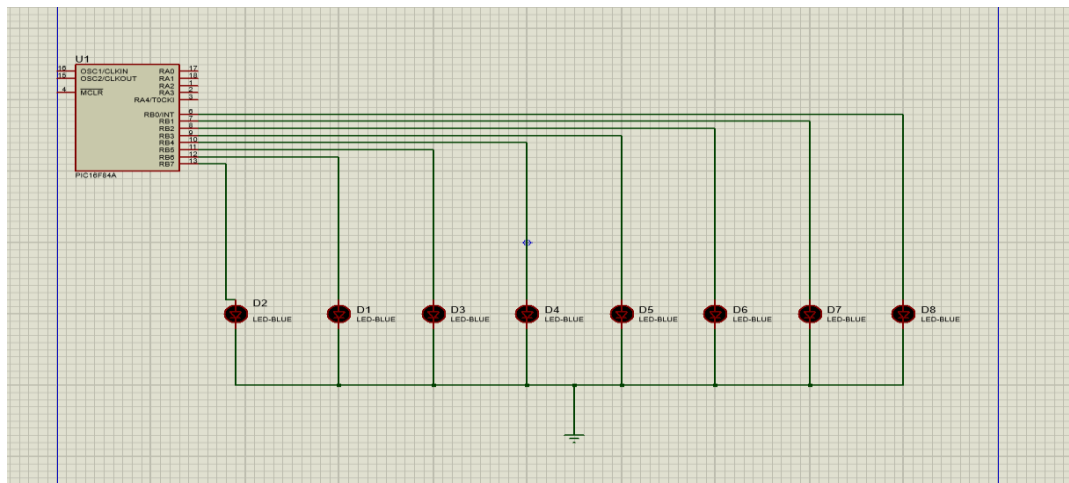


The P type semiconductor is connected with the positive end of battery and N type with the negative end of the battery. When a current is passed through the LED, free electrons from the N layer moves to the P layer in the active region (P-N Junction). This involves a drop from the conduction band to the lower orbit, and electrons release some energy in the form of photons. The frequency and wavelength of the light produced photons is dependent on the material and doping level of the semiconductor. In case of LED, the material used and doping levels are set so as to produce photons with wavelength in the range of desired color of light in the visible spectrum. LEDs have many advantages over incandescent light sources, including lower energy consumption, longer lifetime, improved physical robustness, smaller size, and faster switching.

LEDs uses fall into four major categories:

- Visual signals where light goes more or less directly from the source to the human eye, to convey a message or meaning
- Illumination where light is reflected from objects to give visual response of these objects
- Measuring and interacting with processes involving no human vision.
- Narrow band light sensors where LEDs operate in a reverse-bias mode and respond to incident light, instead of emitting light.

CIRCUIT DIAGRAM:

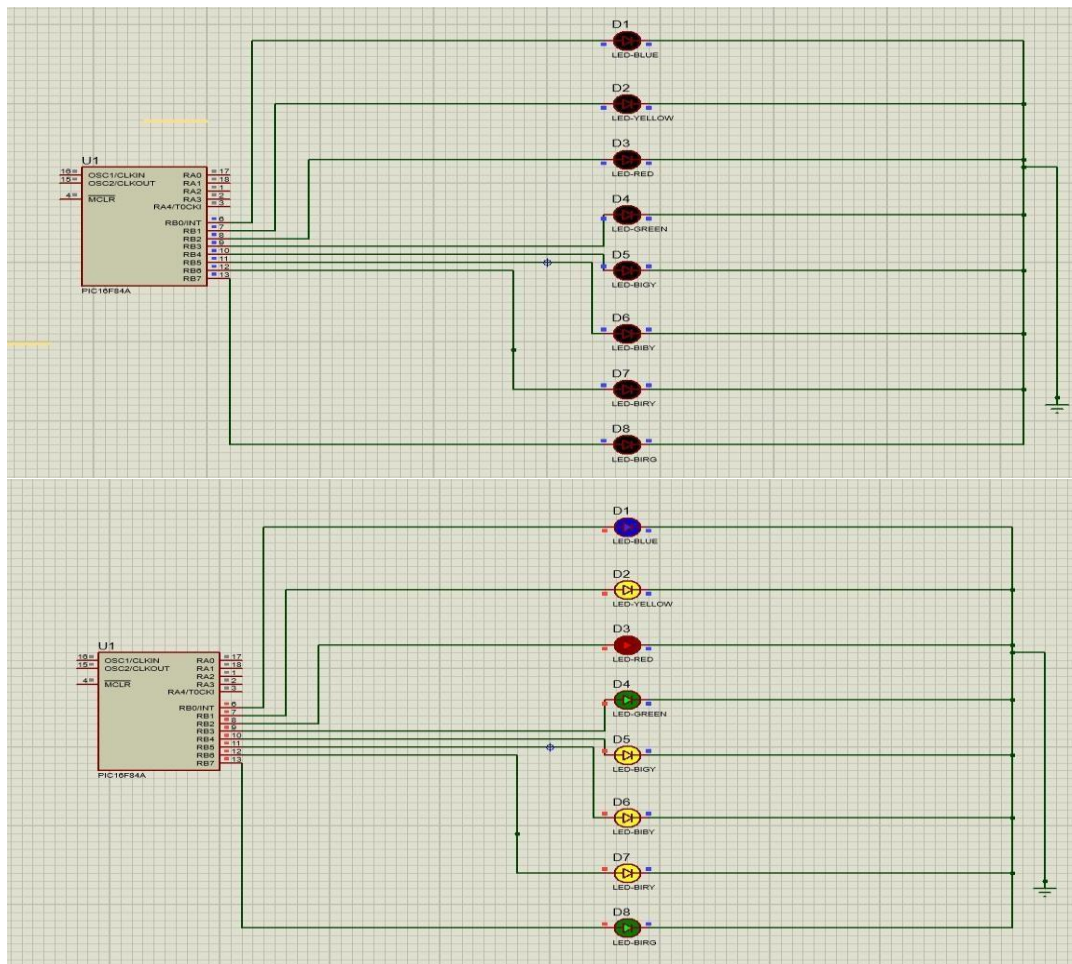


CODE:

```
#include <pic.h>

void main()
{
    int i;
    TRISB = 0;
    while (1)
    {
        PORTB = 0xFF;
        for (i = 0; i < 100; i++)
        {
            PORTB = 0x00;
            for (i = 0; i < 100; i++)
                ;
        }
    }
}
```

OUTPUT:



CONCLUSION: The code worked properly and it was verified using circuit simulation in Proteus such that All 8 LED's blink alternately.

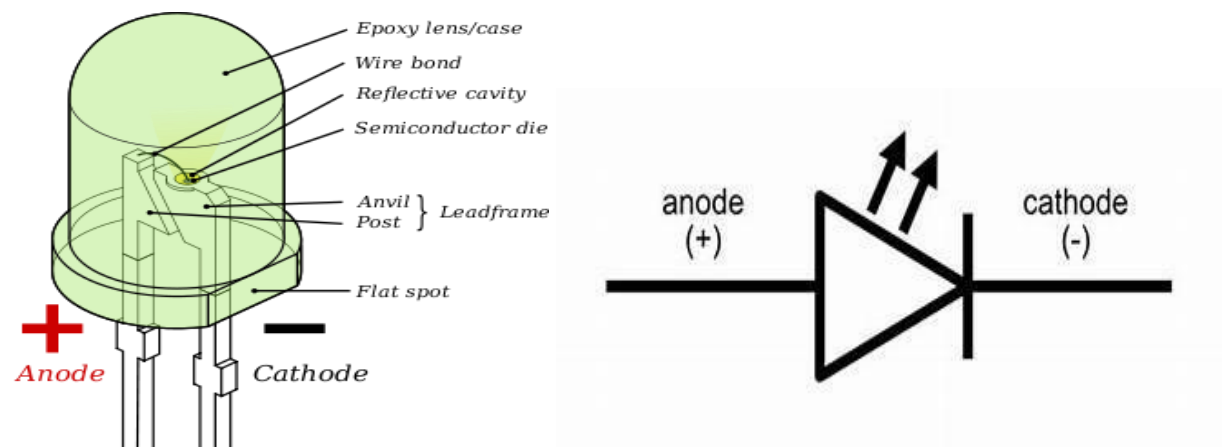
EXPERIMENT-4

AIM: To write an Embedded-C code to blink a set of 8 LEDs interfaced with a PIC microcontroller and verify its working using Proteus such that all 8 LEDs blink alternately.

SOFTWARE: MPLAB X IDE and Proteus 8

THEORY:

A light-emitting diode (LED) is a semiconductor light source that emits light when current flows through it. Electrons in the semiconductor recombine with electron holes, releasing energy in the form of photons. This effect is called electroluminescence. The color of the light (corresponding to the energy of the photons) is determined by the energy required for electrons to cross the band gap of the semiconductor. White light is obtained by using multiple semiconductors or a layer of light-emitting phosphorous on the semiconductor device.



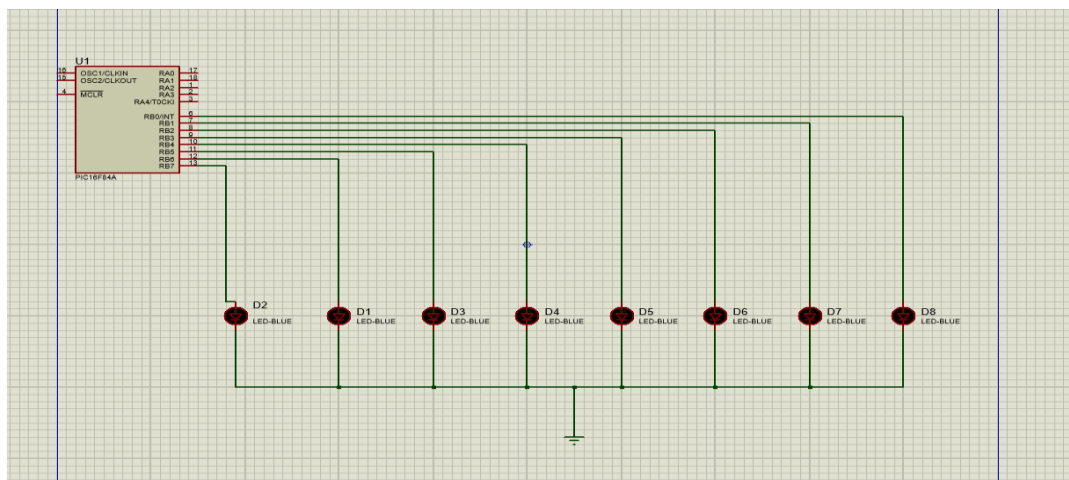
The P type semiconductor is connected with the positive end of battery and N type with the negative end of the battery. When a current is passed through the LED, free electrons from the N layer move to the P layer in the active region (P-N Junction). This involves a drop from the conduction band to the lower orbit and electrons release some energy in the form of photons. The frequency and wavelength of the light produced photons is dependent on the material and doping level of the semiconductor. In case of LED, the material used and doping levels are set so as to produce photons with wavelength in the range of desired color of light in the visible spectrum.

LEDs have many advantages over incandescent light sources, including lower energy consumption, longer lifetime, improved physical robustness, smaller size, and faster switching.

LEDs use fall into four major categories:

- Visual signals where light goes more or less directly from the source to the human eye, to convey a message or meaning.
- Illumination where light is reflected from objects to give visual response of these objects
- Measuring and interacting with processes involving no human vision.
- Narrow band light sensors where LEDs operate in a reverse-bias mode and respond to incident light, instead of emitting light.

CIRCUIT DIAGRAM:



CODE:

```
#include <xc.h>
```

```
#pragma config FOSC = XT // Oscillator Selection bits (XT oscillator)
```

```
#pragma config WDTE = OFF // Watchdog Timer (WDT disabled)
```

```
#pragma config PWRTE = ON // Power-up Timer Enable bit (Power-up Timer is enabled)
```

```
#pragma config CP = OFF // Code Protection bit (Code protection disabled)
```

```
#define _XTAL_FREQ      \
    4000000 void main(void) \
    {
```

```
    TRISB = 0X00;
```

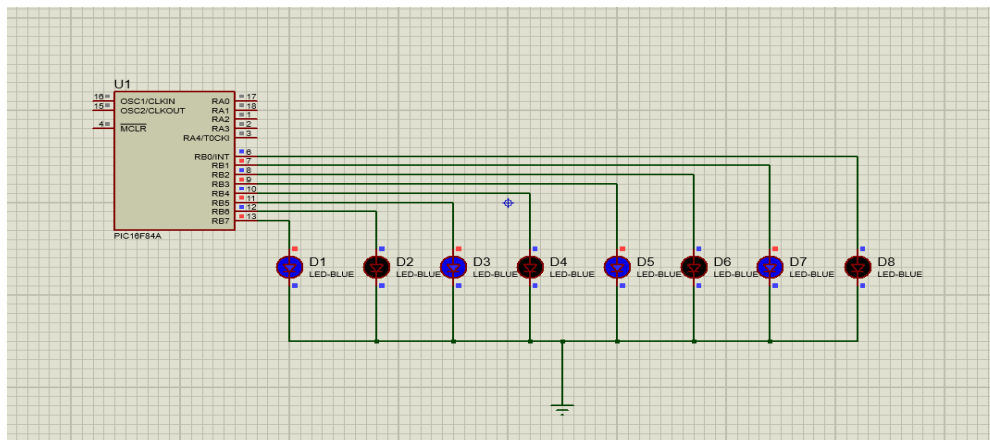
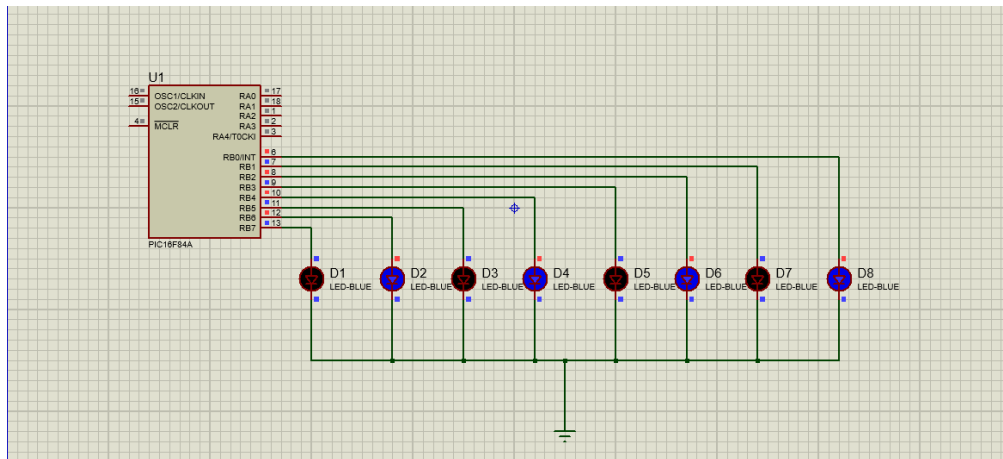
```
    PORTB = 0X00;
```

```

while (1)
{
    PORTB = 0b10101010;
    delay_ms(1000);
    PORTB = 0b01010101;
    delay_ms(1000);
}
}

```

OUTPUT:



CONCLUSION: The code worked properly and it was verified using circuit simulation in Proteus such that all 8 LEDs blink simultaneously.

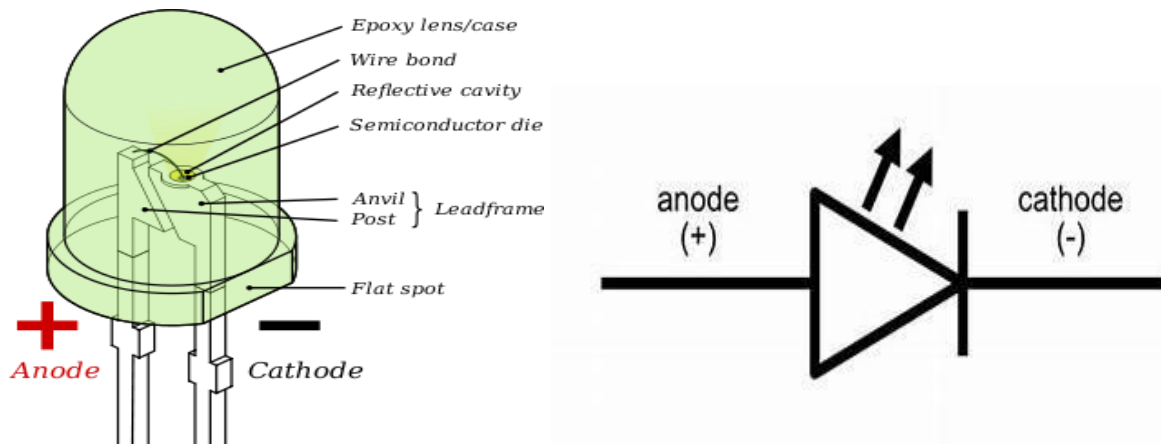
EXPERIMENT-5

AIM: To write an Embedded-C code to blink a set of 8 LEDs interfaced with a PIC microcontroller and verify its working using Proteus such that all 8 LEDs blink one by one.

SOFTWARE: MPLAB X IDE and Proteus 8

THEORY:

A light-emitting diode (LED) is a semiconductor light source that emits light when current flows through it. Electrons in the semiconductor recombine with electron holes, releasing energy in the form of photons. This effect is called electroluminescence. The color of the light (corresponding to the energy of the photons) is determined by the energy required for electrons to cross the band gap of the semiconductor. White light is obtained by using multiple semiconductors or a layer of light-emitting phosphorous on the semiconductor device.



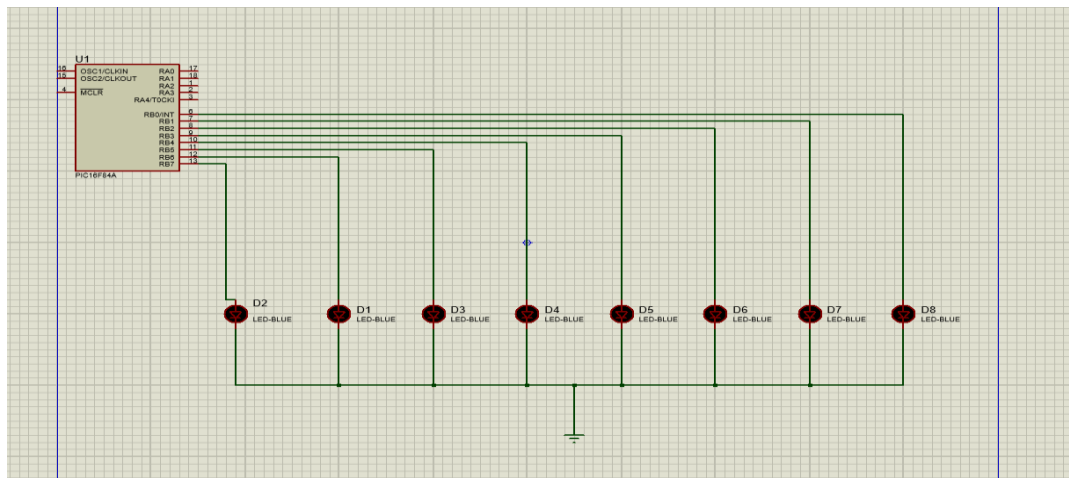
The P type semiconductor is connected with the positive end of battery and N type with the negative end of the battery. When a current is passed through the LED, free electrons from the N layer moves to the P layer in the active region (P-N Junction). This involves a drop from the conduction band to the lower orbit and electrons release some energy in the form of photons. The frequency and wavelength of the light produced photons is dependent on the material and doping level of the semiconductor. In case of LED, the material used and doping levels are set so as to produce photons with wavelength in the range of desired color of light in the visible spectrum.

LEDs have many advantages over incandescent light sources, including lower energy consumption, longer lifetime, improved physical robustness, smaller size, and faster switching.

LEDs use fall into four major categories:

- Visual signals where light goes more or less directly from the source to the human eye, to convey a message or meaning.
- Illumination where light is reflected from objects to give visual response of these objects
- Measuring and interacting with processes involving no human vision.
- Narrow band light sensors where LEDs operate in a reverse-bias mode and respond to incident light, instead of emitting light.

CIRCUIT DIAGRAM:



CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <xc.h>
// CONFIG
#pragma config FOSC = XT // Oscillator Selection bits (XT oscillator)
#pragma config WDTE = OFF // Watchdog Timer (WDT disabled)
#pragma config PWRTE = ON // Power-up Timer Enable bit (Power-up Timer is enabled)
#pragma config CP = OFF // Code Protection bit (Code protection disabled)

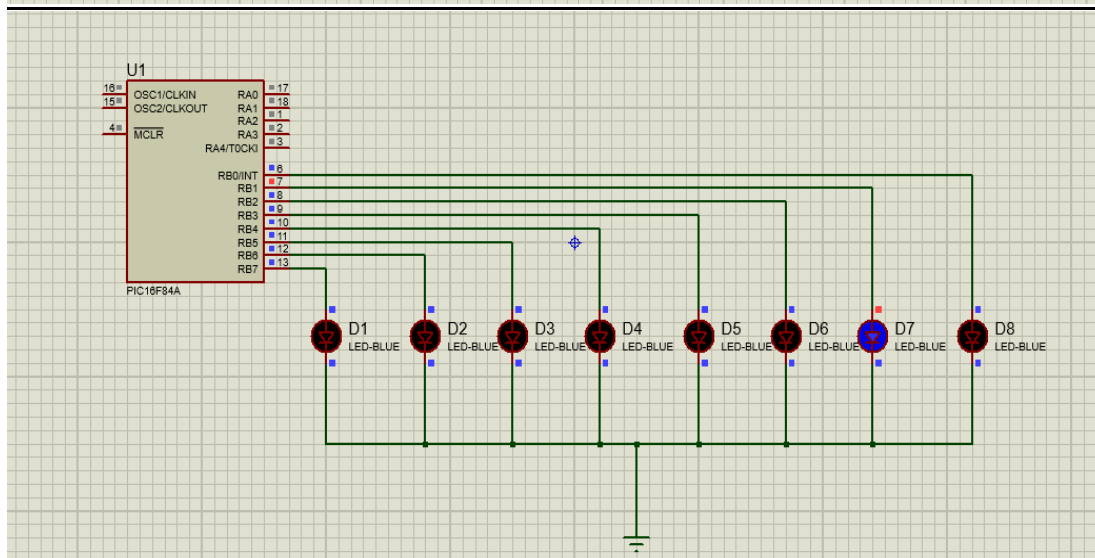
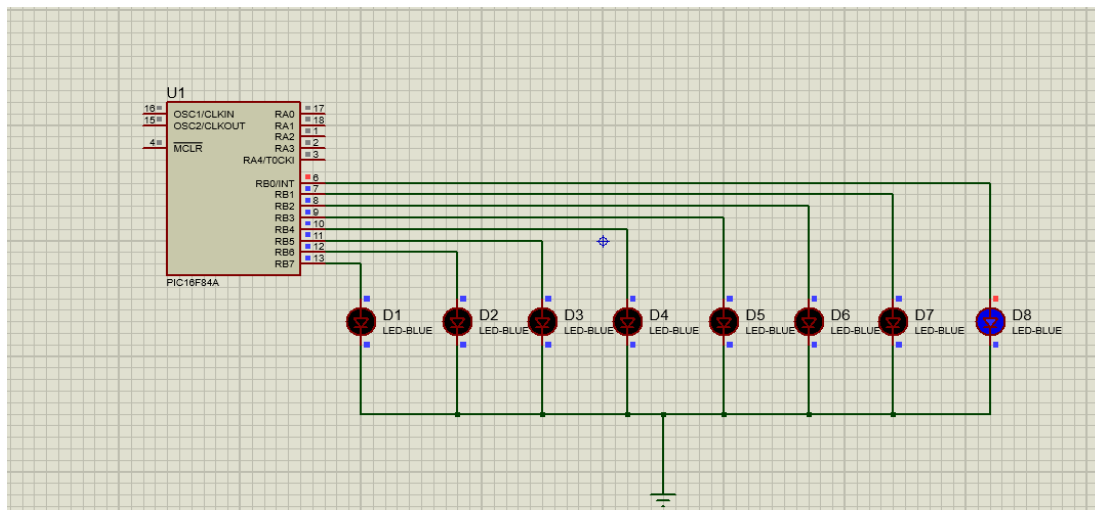
#define _XTAL_FREQ 4000000
// #pragma config statements should precede project file includes.
// Use project enums instead of #define for ON and OFF.
```

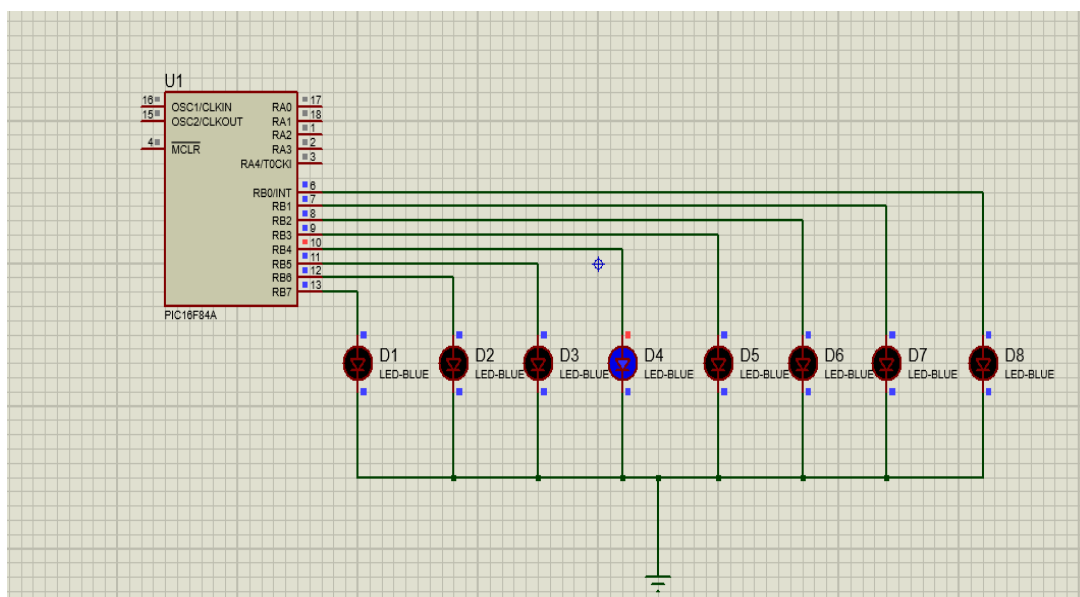
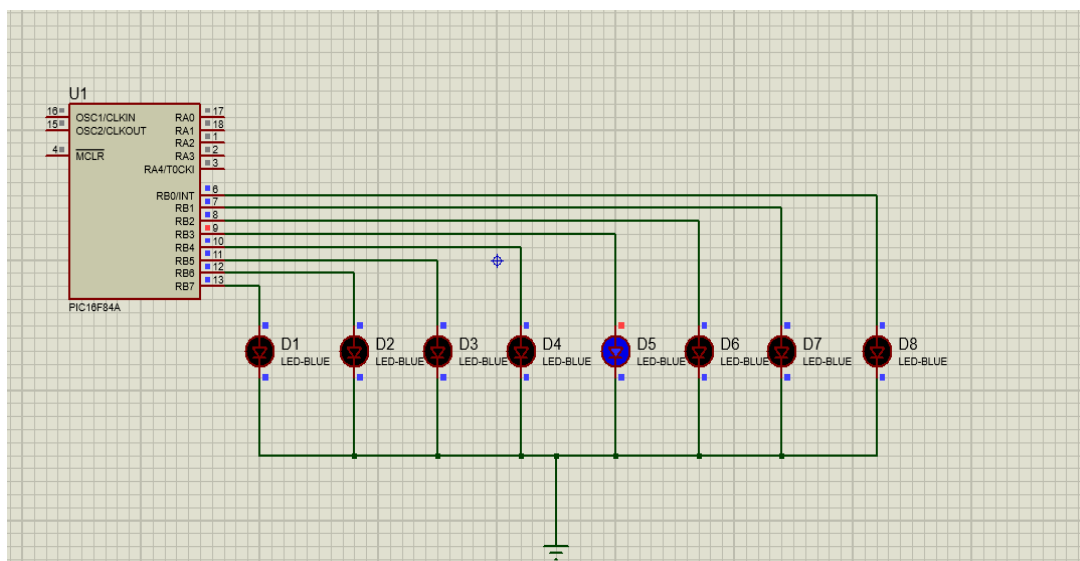
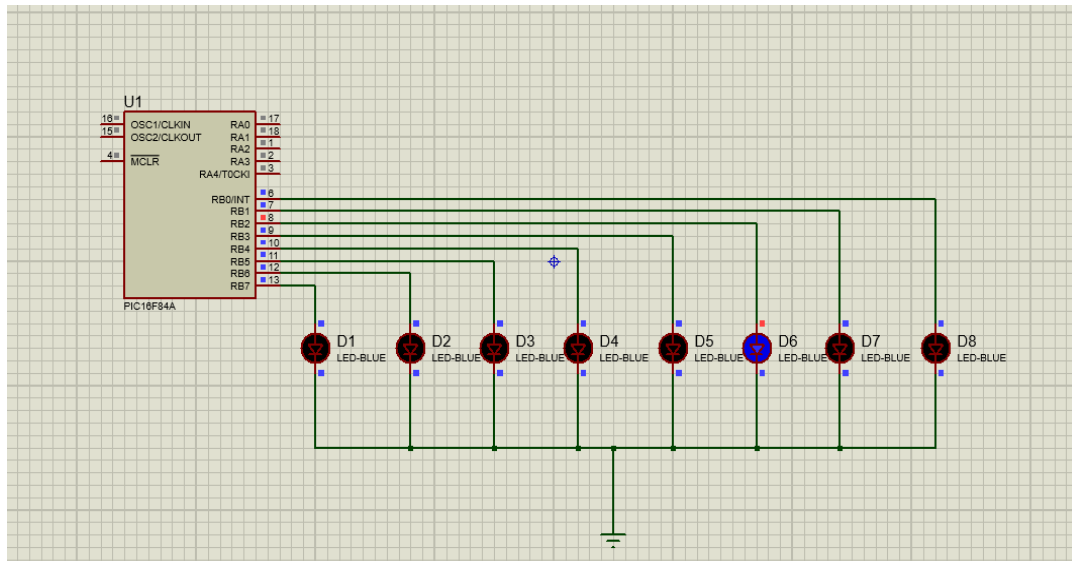
```

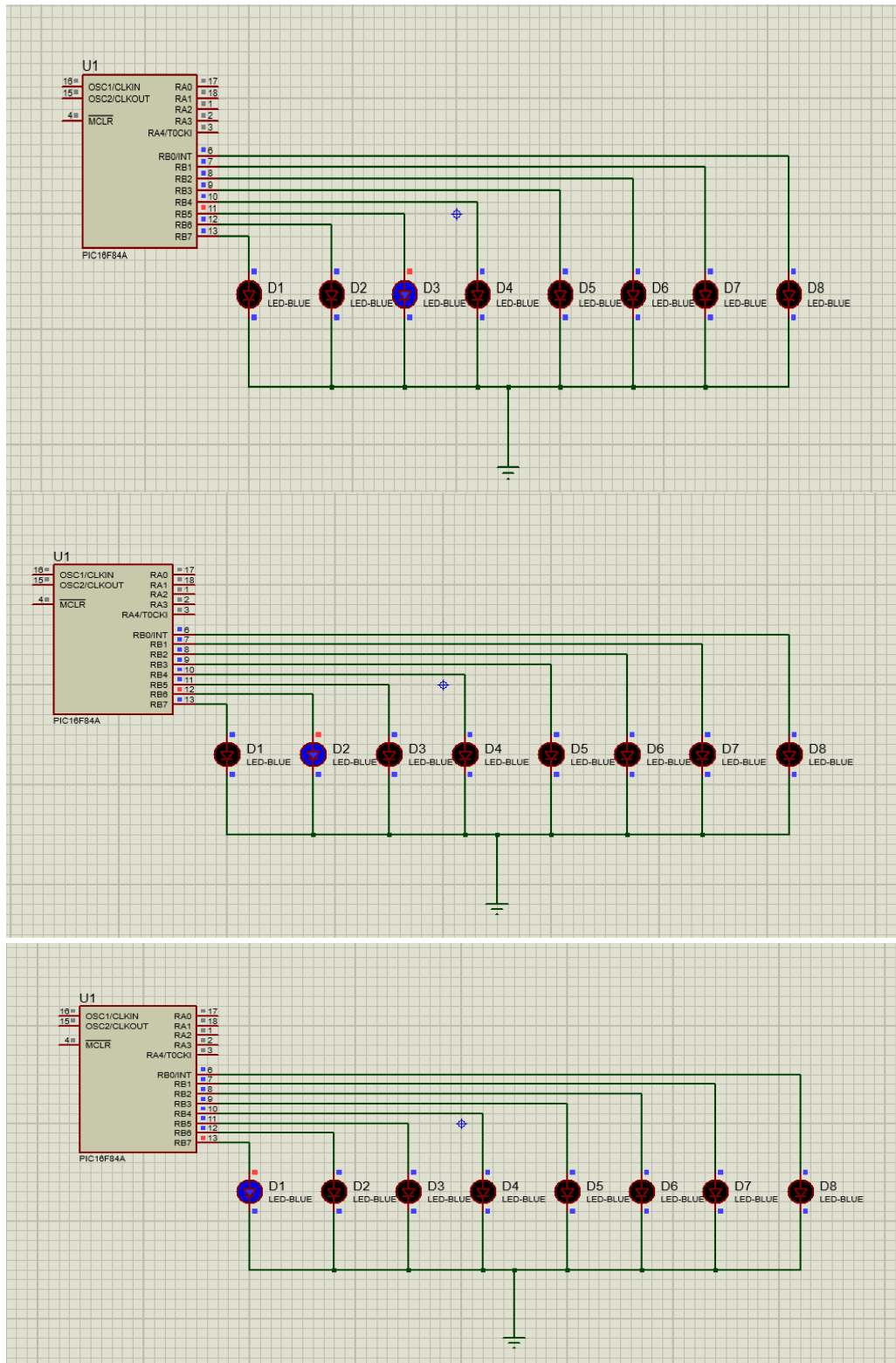
void main(void)
{
    TRISB = 0x00;
    PORTB = 0x00;
    unsigned char i;
    while (1)
    {
        for (i = 0; i < 8; i++)
        {
            PORTB = (1 << i);
            delay_ms(1000);
        }
    }
}

```

OUTPUT:







CONCLUSION: The code worked properly and it was verified using circuit simulation in Proteus such that all 8 LEDs blink one by one.

EXPERIMENT-6

AIM: To write an Embedded-C code to display roll number on seven segment display interfaced with PIC16F877a microcontroller and verify its working using Proteus.

SOFTWARE: MPLAB X IDE and Proteus 8

THEORY:

A seven-segment display (SSD), or seven-segment indicator, is a form of electronic display device for displaying decimal numerals that is an alternative to the more complex dot matrix displays. Seven-segment displays are widely used in digital clocks, electronic meters, basic calculators, and other electronic devices that display numerical information.

The displays common pin is generally used to identify which type of 7-segment display it is. As each LED has two connecting pins, one called the “Anode” and the other called the “Cathode”, there are therefore two types of LED 7-segment display called: **Common Cathode (CC)** and **Common Anode (CA)**.

The difference between the two displays, as their name suggests, is that the common cathode has all the cathodes of the 7-segments connected directly together and the common anode has all the anodes of the 7-segments connected together.

Common Cathode 7-segment Display:

The Common Cathode (CC) – In the common cathode display, all the cathode connections of the LED segments are joined together to logic “0” or ground. The individual segments are illuminated by application of a “HIGH”, or logic “1” signal via a current limiting resistor to forward bias the individual Anode terminals (a-g).

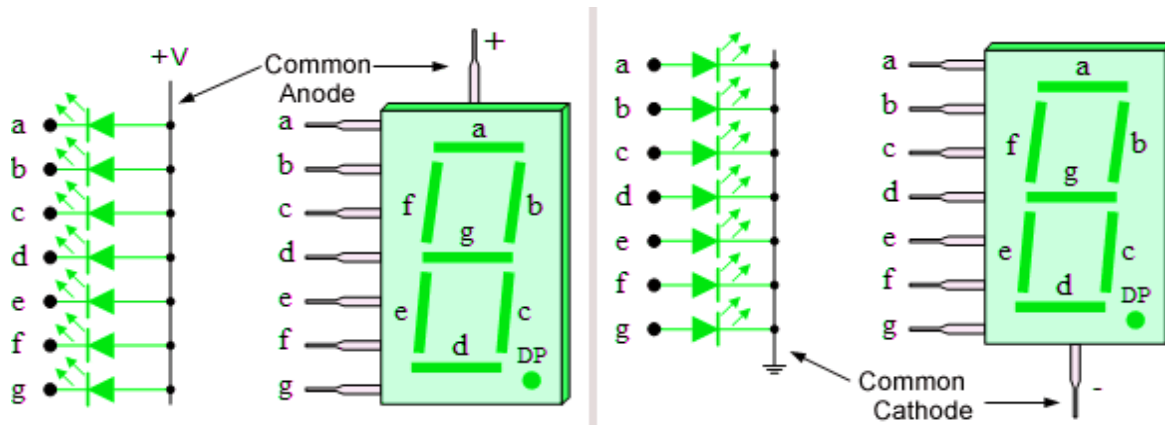
Common Anode 7-segment Display:

The Common Anode (CA) – In the common anode display, all the anode connections of the LED segments are joined together to logic “1”. The individual segments are illuminated by applying a ground, logic “0” or “LOW” signal via a suitable current limiting resistor to the Cathode of the particular segment (a-g).

In general, common anode displays are more popular as many logic circuits can sink more current than they can source. Also note that a common cathode display is not a direct

replacement in a circuit for a common anode display and vice versa, as it is the same as connecting the LEDs in reverse, and hence light emission will not take place.

Depending upon the decimal digit to be displayed, the particular set of LEDs is forward biased. For instance, to display the numerical digit 0, we will need to light up six of the LED segments corresponding to a, b, c, d, e, and f. Thus the various digits from 0 through 9 can be displayed using a 7-segment.



Numbers	Common Cathode		Common Anode	
	(DP)GFEDCBA	HEX Code	(DP)GFEDCBA	HEX Code
0	00111111	0x3F	11000000	0xC0
1	00000110	0x06	11111001	0xF9
2	01011011	0x5B	10100100	0xA4
3	01001111	0x4F	10110000	0xB0
4	01100110	0x66	10011001	0x99
5	01101101	0x6D	10010010	0x92
6	01111101	0x7D	10000010	0x82
7	00000111	0x07	11111000	0xF8
8	01111111	0x7F	10000000	0x80
9	01101111	0x6F	10010000	0x90

CIRCUIT DIAGRAM:

1. Common Cathode:

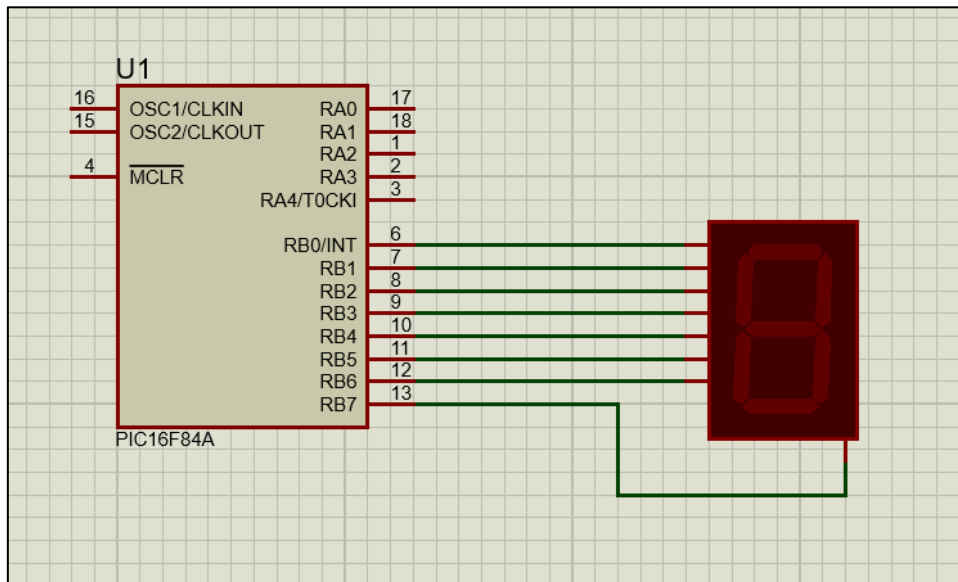


Fig: Common Cathode Interface using PIC Microcontroller

2. Common Anode:

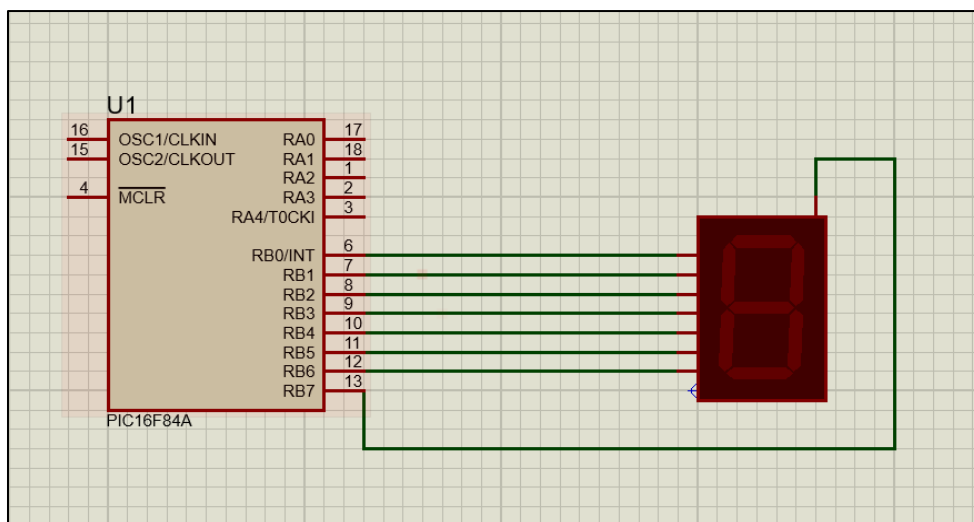


Fig: Common Anode Interface using PIC Microcontroller

CODE:

1. Common Cathode:

```
#include <xc.h>
#define _XTAL_FREQ 4000000 unsigned char segment_code[] = {
0x3F, // 0 (Segments a,b,c,d,e,f)
0x06, // 1 (Segments b,c)
0x5B, // 2 (Segments a,b,g,e,d)
0x4F, // 3 (Segments a,b,c,d,g)
0x66, // 4 (Segments f,g,b,c)
```



```

    0x6D, // 5 (Segments a,f,g,c,d)
    0x7D, // 6 (Segments a,f,g,e,c,d)
    0x07, // 7 (Segments a,b,c)
    0x7F, // 8 (All segments on)
    0x6F // 9 (a,b,c,d,f,g)
}
;
void main(void)
{
    TRISB = 0x00; // Set all Port B pins as Outputs
    PORTB = 0x00; // Start with the display cleared
    while (1)
    {
        for (int i = 0; i < 10; i++)
        {
            PORTB = segment_code[i]; // Send hex code to display
            delay_ms(200);           // Wait 1 second before next number
        }
        // After reaching 9, the loop starts over at 0 automatically
    }
}

```

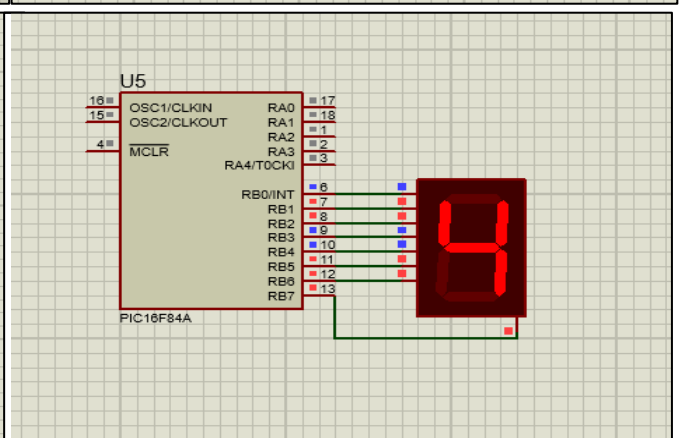
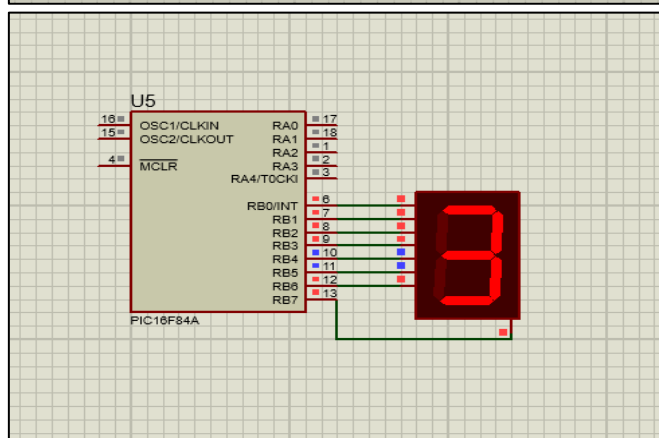
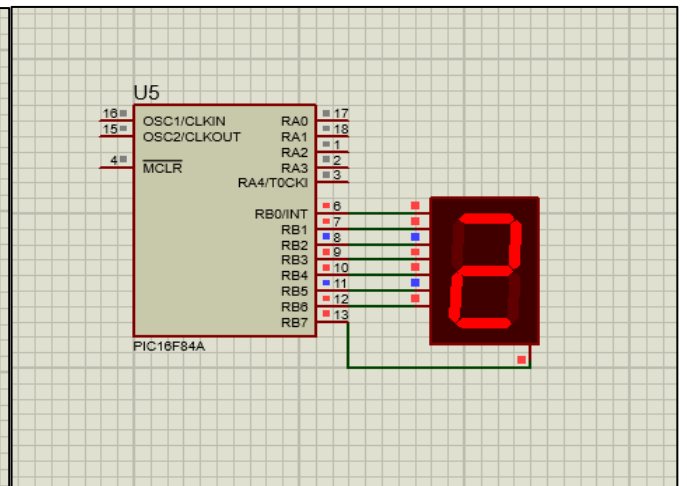
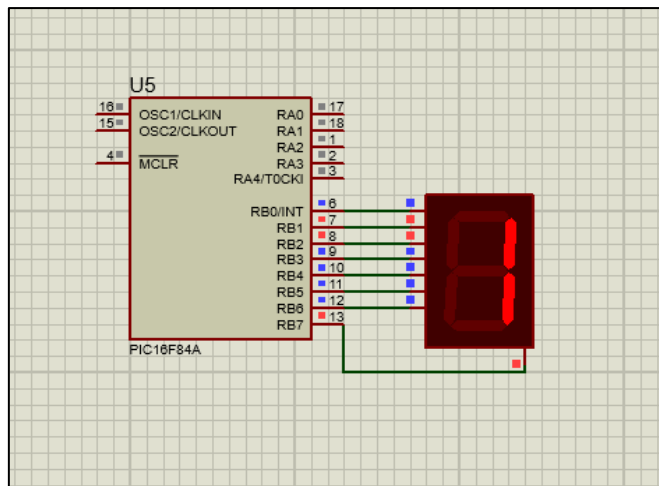
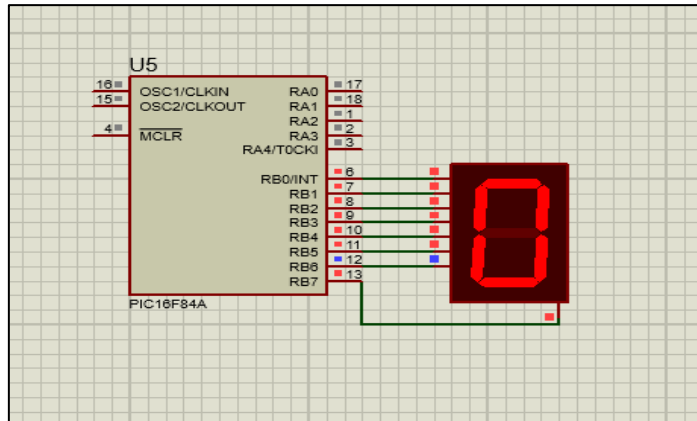
2. Common Anode:

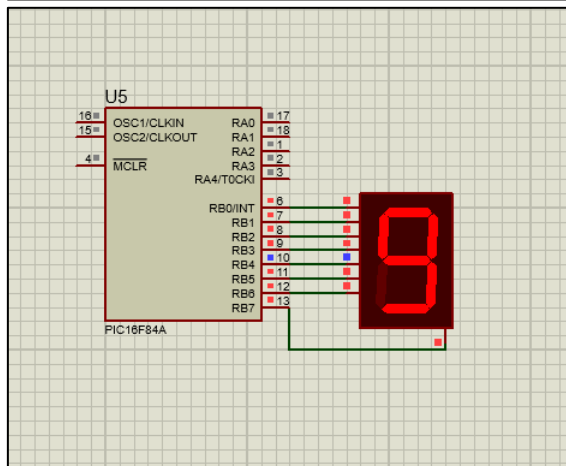
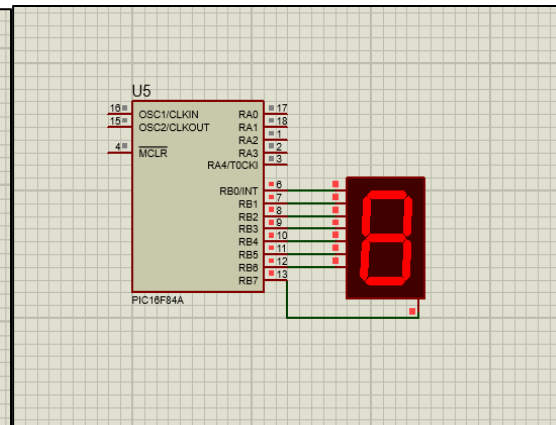
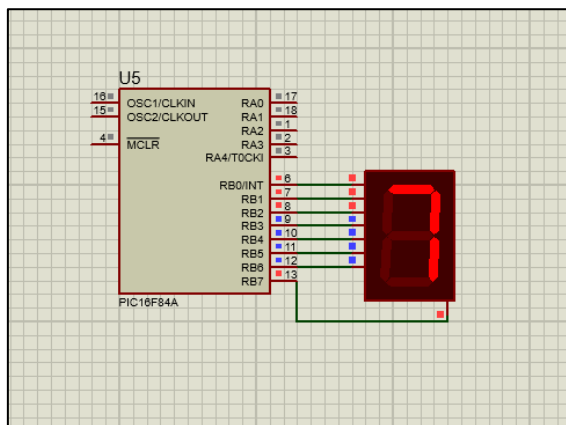
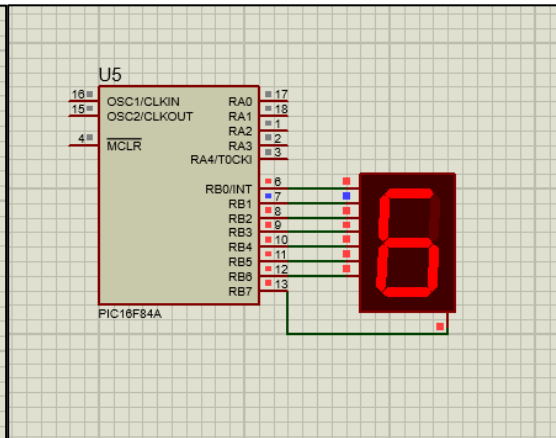
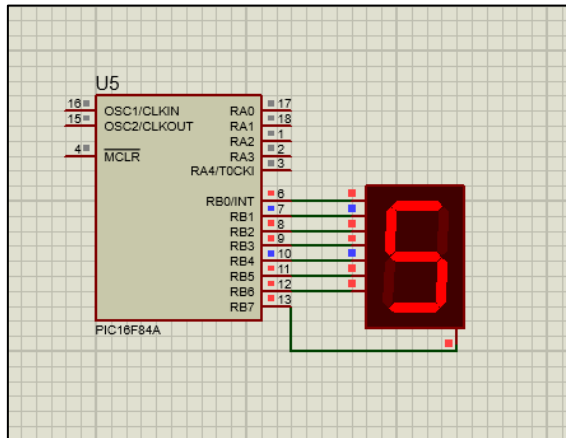
```

#include <xc.h>
#define _XTAL_FREQ 4000000
unsigned char segment_code_anode[] = {
    0xC0, // 0
    0xF9, // 1
    0xA4, // 2
    0xB0, // 3
    0x99, // 4
    0x92, // 5
    0x82, // 6
    0xF8, // 7
    0x80, // 8
    0x90 // 9
};
void main(void)
{
    TRISB = 0x00; // Set all Port B pins as Outputs
    PORTB = 0x00; // Start with the display cleared
    while (1)
    {
        for (int i = 0; i < 10; i++)
        {
            PORTB = segment_code_anode[i]; // Send hex code to display
            delay_ms(200);                 // Wait 1 second before next number
        }
        // After reaching 9, the loop starts over at 0 automatically
    }
}

```

OUTPUT:





CONCLUSION: The code has worked properly, and it was verified using circuit simulation in Proteus such that digit 0-9 has been displayed using following configurations:

- Common Cathode
- Common Anode

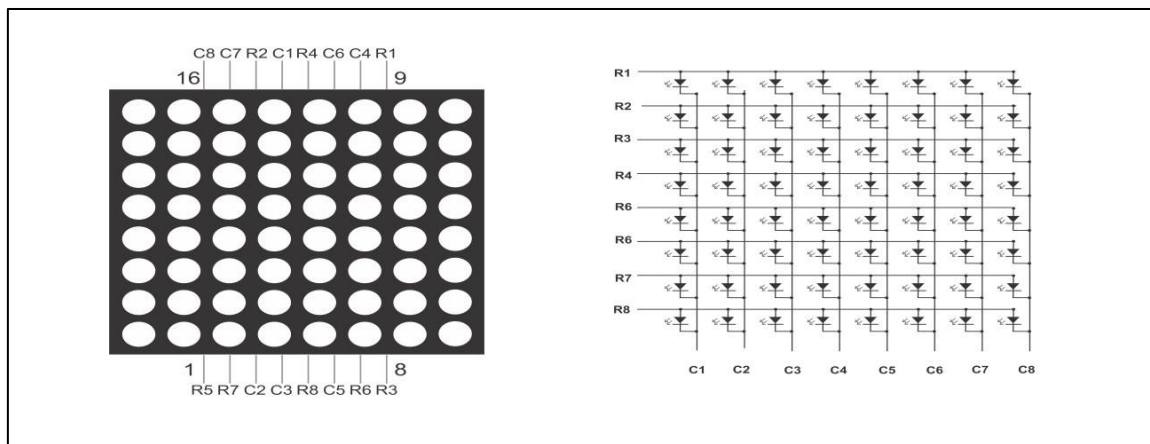
EXPERIMENT-7

AIM: To write an Embedded C code to display A, B, C on 5X7 LED Matrix display interfaced with PIC microcontroller and verify its working using Proteus

SOFTWARE: MPLAB X IDE and Proteus 8

THEORY:

An LED dot matrix display consists of a matrix of LEDs arranged in a rectangular configuration. The LEDs are connected in rows and columns, allowing individual LEDs to be turned ON or OFF by selecting the appropriate row and column combination. This structure reduces the number of required I/O pins compared to controlling each LED separately, making it efficient for microcontroller-based systems. In this experiment, a 5x7 LED matrix display is interfaced with a PIC microcontroller to display characters A and C. The figure below shows a typical 5x7 LED matrix configuration. By controlling the row and column lines appropriately, specific patterns can be generated to display desired characters. The display works on the principle of multiplexing, where rows are activated one at a time, and the corresponding column data is provided to illuminate the required LEDs. Rapid switching between rows creates the illusion of a continuous display due to persistence of vision.



WORKING:

Light Emitting Diodes (LEDs) provide a cheap and convenient way to display information electronically. LED dot matrix displays are widely used in various electronic devices for showing characters and symbols. In a 5x7 matrix display, 5 columns and 7 rows of LEDs

are arranged in a grid format. By selectively turning ON the required LEDs in a specific pattern, different characters can be formed. In this experiment, the PIC microcontroller sends appropriate signals to the row and column pins to display the characters A and C. The multiplexing technique ensures that the display appears steady even though each row is activated sequentially for a very short duration.

CIRCUIT DIAGRAM:

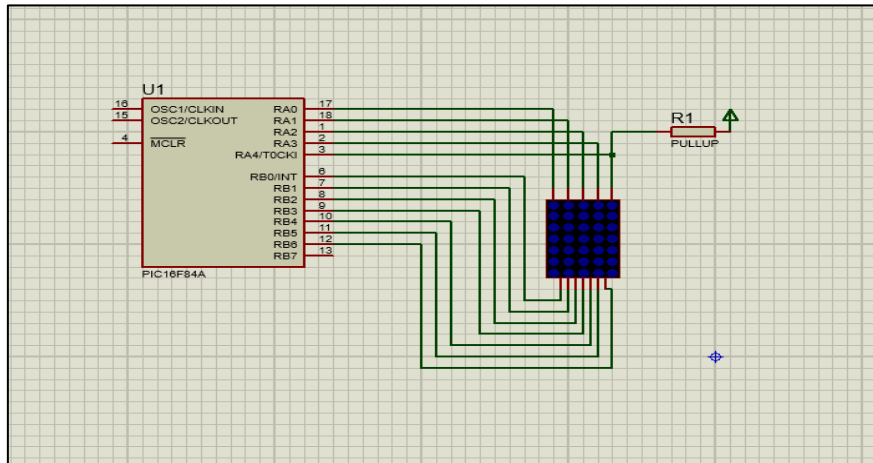


Fig: 5X7 Display LED Interface using PIC Microcontroller

CODE:

```
#include <xc.h>
// CONFIG BITS
#pragma config FOSC = XT
#pragma config WDTE = OFF
#pragma config PWRTE = ON
#pragma config CP = OFF
#define _XTAL_FREQ 4000000
// Character patterns (Column wise)
// Since matrix is COMMON ANODE → rows must be LOW to glow
const unsigned char A[5] = {
    0b10000001,
    0b11101110,
    0b11101110,
    0b11101110,
    0b10000001};
const unsigned char B[5] = {0b10000000,
    0b10110110,
    0b10110110,
    0b10110110,
    0b11001001};
```

```

const unsigned char C[5] = {0b11000001,
                             0b10111110,
                             0b10111110,
                             0b10111110,
                             0b11011101};

void display(const unsigned char *ch)
{
    for (int refresh = 0; refresh < 150; refresh++)
    {
        for (int col = 0; col < 5; col++)
        {
            PORTA = 0x00;    // Disable all columns
            PORTB = ch[col];  // Send inverted row data
            PORTA = (1 << col); // Enable one column (HIGH active)
            delay_ms(2);
        }
    }
}

void main()
{
    TRISA = 0x00; // Columns
    TRISB = 0x00; // Rows
    PORTA = 0x00;
    PORTB = 0xFF;
    while (1)
    {
        display(A);
        display(B);
        display(C);
    }
}

```

OUTPUT:

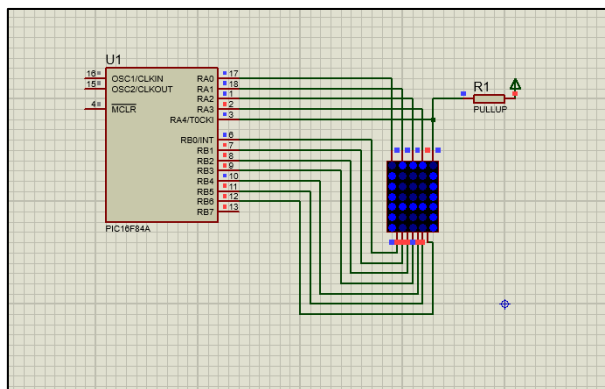


Fig: Displaying A using LED display

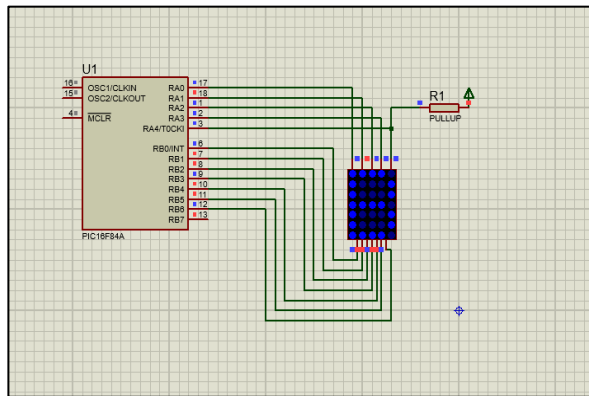


Fig: Displaying B using LED display

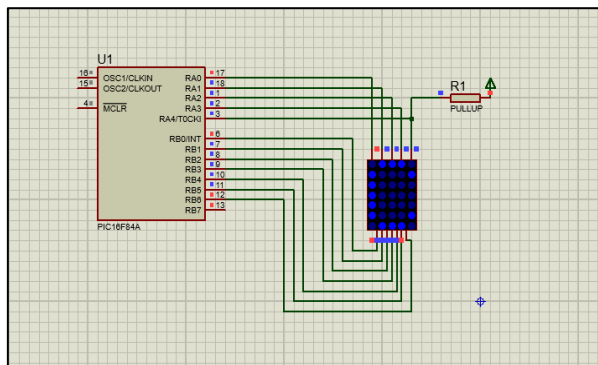


Fig: Displaying C using LED display

CONCLUSION: The code worked properly, and it was verified using circuit simulation in Proteus such that A-C have been displayed successfully.

EXPERIMENT-8

AIM: To write an Embedded-C code to display 2 digit number on 8x8 LED matrix display interfaced with a PIC microcontroller and verify its working using Proteus.

SOFTWARE & HARDWARE: * MPLAB X IDE

- Proteus 8 Professional
- PIC16F877 Microcontroller (Target Device)
- 8x8 LED Matrix

THEORY: An 8x8 LED matrix consists of 64 Light Emitting Diodes (LEDs) arranged in an 8-row by 8-column grid. Controlling 64 individual LEDs would normally require 64 distinct microcontroller pins. To solve this, the matrix uses a technique called **multiplexing**. The anodes of all LEDs in a specific row are connected together, and the cathodes of all LEDs in a specific column are connected together (or vice versa, depending on whether it is a row-anode or row-cathode matrix).

Because the PIC16F877 has sufficient I/O pins, we can dedicate one entire port (like PORTB) to control the rows and another port (like PORTD) to scan the columns.

Displaying Two Digits on a Single Matrix: A standard font usually takes up 5x7 or 8x8 pixels for a single character. To display a two-digit number simultaneously on a single 8x8 matrix without scrolling, a compressed **3x5 pixel font** is highly effective.

- The first digit uses the first 3 columns.
- The 4th column is left blank as a spacer.
- The second digit uses the next 3 columns.
- The 8th column is left blank.

By rapidly scanning through the 8 columns one by one (activating the column and sending the corresponding row pattern) at a high frequency, the human eye perceives both digits as a single, continuous, static image due to the **persistence of vision**.

CIRCUIT DIAGRAM CONNECTIONS:

- **PORTB (RB0 - RB7):** Connected to the 8 Row pins of the 8x8 LED Matrix.
- **PORTD (RD0 - RD7):** Connected to the 8 Column pins of the 8x8 LED Matrix.
- **Crystal Oscillator:** A 4MHz crystal connected across OSC1 and OSC2 with 22pF ceramic capacitors.

CODE:

```
#include <xc.h>

// CONFIGURATION BITS
#pragma config FOSC = XT    // Oscillator Selection bits (XT oscillator)
#pragma config WDTE = OFF   // Watchdog Timer Enable bit (WDT disabled)
#pragma config PWRTE = ON   // Power-up Timer Enable bit (PWRT enabled)
#pragma config CP = OFF     // Flash Program Memory Code Protection bit

#define _XTAL_FREQ 4000000

const unsigned char display_pattern[8] = {
    0b00010001, // Col 0: '3' left
    0b00010101, // Col 1: '3' middle
    0b00011111, // Col 2: '3' right
    0b00000000, // Col 3: Space
    0b00011001, // Col 4: '7' left
    0b00000101, // Col 5: '7' middle
    0b00000011, // Col 6: '7' right
    0b00000000 // Col 7: Space
};

void main(void) {
    TRISB = 0x00; // Set PORTB as output (Controls Rows)
    TRISD = 0x00; // Set PORTD as output (Controls Columns)

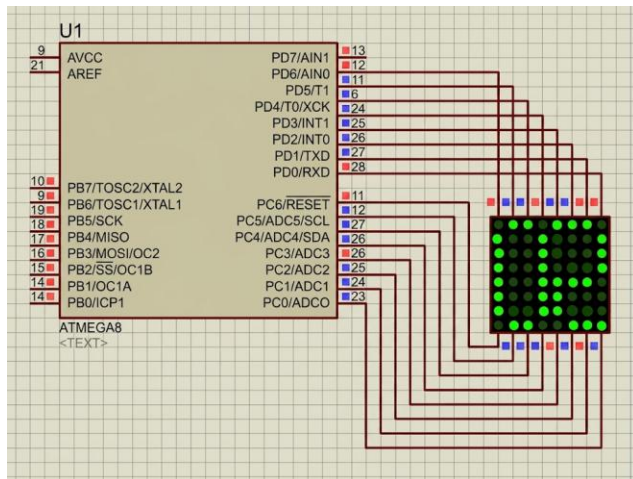
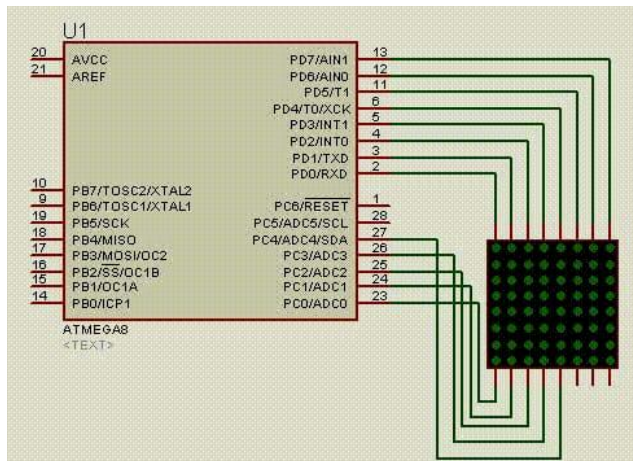
    PORTB = 0x00; // Initialize Ports
    PORTD = 0xFF;

    while(1) {
        // Multiplexing loop to scan columns
        for(int col = 0; col < 8; col++) {

            PORTD = 0xFF;          // Disable all columns to prevent ghosting
            PORTB = display_pattern[col]; // Feed the active row pattern
            PORTD = ~(1 << col);    // Activate the current column (Active LOW)

            __delay_ms(2);          // Short delay for persistence of vision
        }
    }
}
```

OUTPUT:



CONCLUSION: The Embedded-C code was successfully written and compiled for the PIC16F877. The multiplexing logic worked properly, and it was verified using circuit simulation in Proteus such that a static 2-digit number was clearly displayed side-by-side on a single 8x8 LED matrix.